



Leibniz Institute
for high
performance
microelectronics

Welcome to the IHP OS Analog Certificate Course

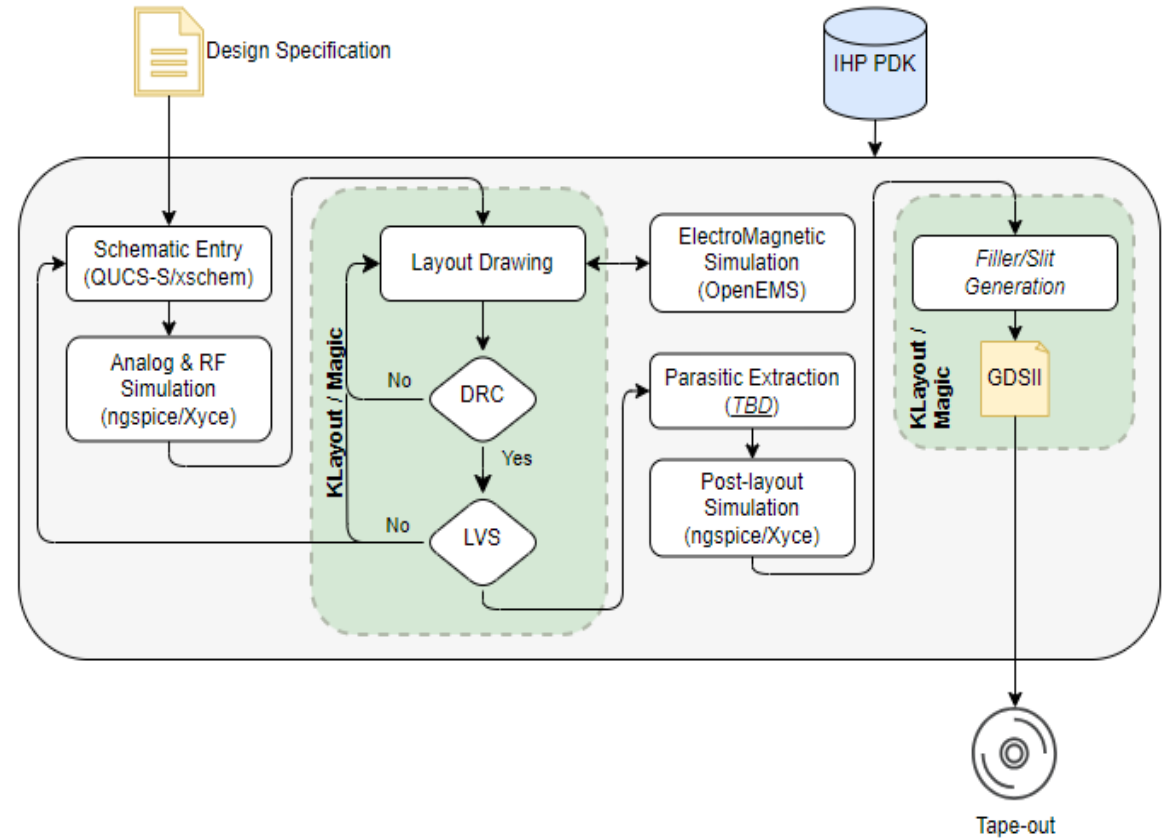
Phillip Ferreira Baade–Pedersen (Design Engineer)
Erlangen - 23/02/2026

Projects: BMBF -> FMD-QNC (16ME0831)

About The Scope Of This Course



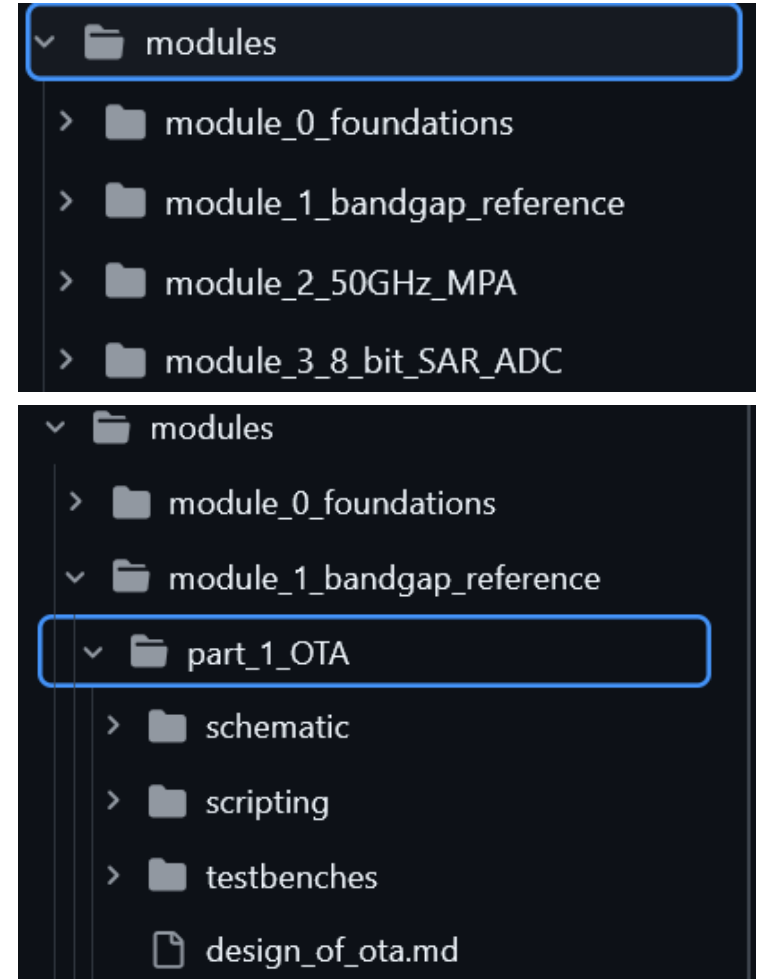
- 0 Navigating in the analog OS echo system
- 0 Going from design to layout
- 0 Setting up a professional workflow similar to conventional chip design
- 0 Understand the gaps yet to be closed
- ~~-0 Design Circuits~~



The Structure of the Course:



- 0 Module based course
- 0 All content is available in the repository on your machines
- 0 Use the markdown files as cheat sheets or if you get stuck
- 0 Most of the designs should run with no additional setup
- 0 Its recommended to copy the structure seen in the repo (alternatively create duplicated folders)





Leibniz Institute
for high
performance
microelectronics

Live Demonstration Of The Repository

Now, Let's Get to Know Each Other!



- 0 Have you worked with any of the tools we'll be using?
- 0 What challenges have you faced in similar projects?
- 0 What's your background and experience in this field?
- 0 What are you hoping to learn from this course?

IHP in a nutshell



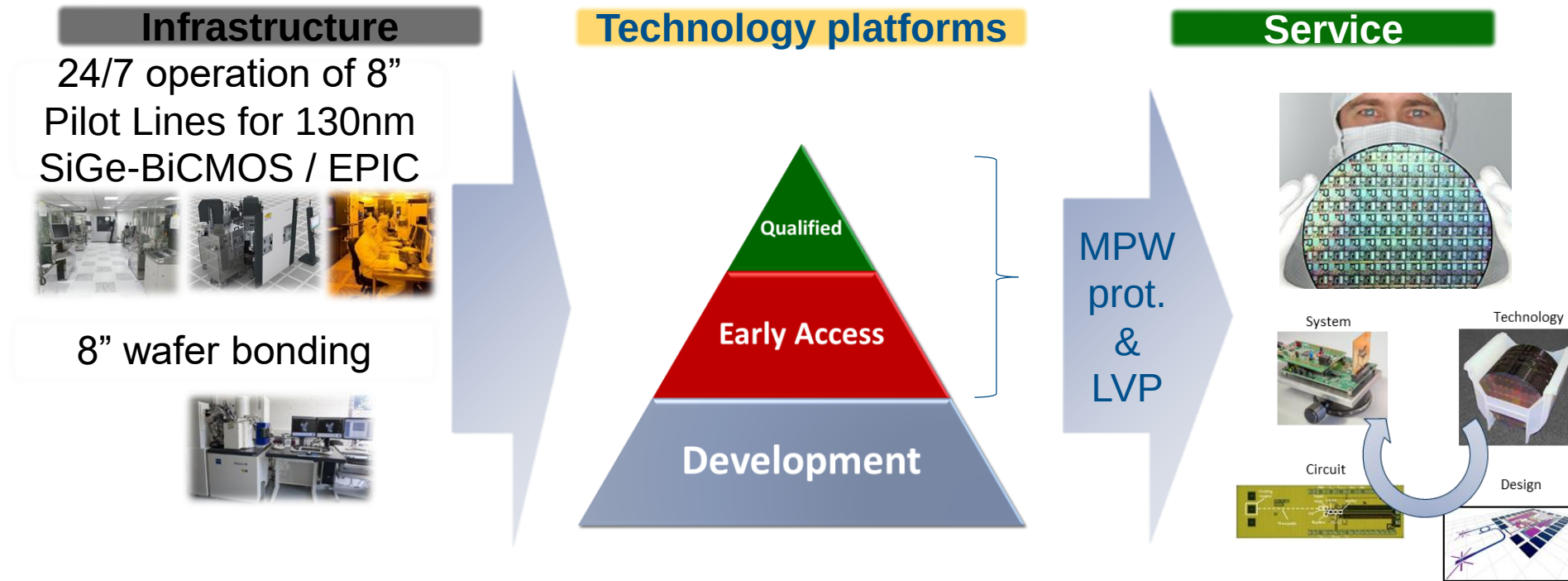
- 0 IHP is the European research and innovation centre for silicon-based systems, ultrahigh-frequency circuits and technologies,
- 0 Unique selling point of a 200mm pilot line for state-of-the-art BiCMOS technologies, operated under industry-like conditions, 24/7, for the provision of prototypes and low-volume production runs.
- 0 Qualified technological platform with direct access for science and industry
- 0 Vertical structure from material research to system architecture
- 0 350+ employees, 40+ nationalities



Vision

"We create foundations and prototype applications based on future silicon-based technologies and systems for a digitalized and networked world as well as for the sustainable preservation of our natural living conditions."

μ & nano - Technologies for wireless and broadband communication, health, security, space and industrial automation



130nm SiGe BiCMOS Technologies for RF Applications

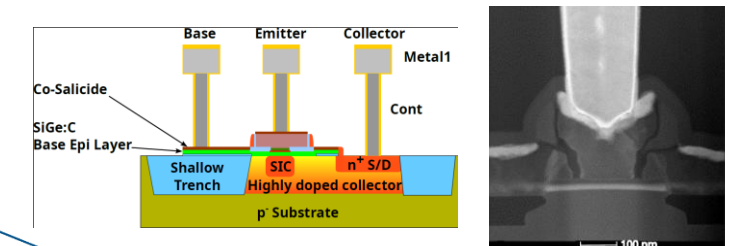


	SG13S	SG13G2	SG13G3
HBT f_t / f_{max}	250 / 340 GHz	350 / 450 GHz	470 / 650 GHz
$W_{Emitter}$	170 nm	130 nm	110 nm
HBT BV_{CE0}	1.7 V	1.6 V	1.5 V
CMOS node		130 nm	
Active devices	Schottky diode, Antenna diodes, PN diodes, ESD		
Varactors		NMOS Varactor	
Resistors	Poly-Si, Thin Film		Poly-Si
MIM Caps	1.5 fF / μm^2	1.5 fF / μm^2	2.1 fF / μm^2

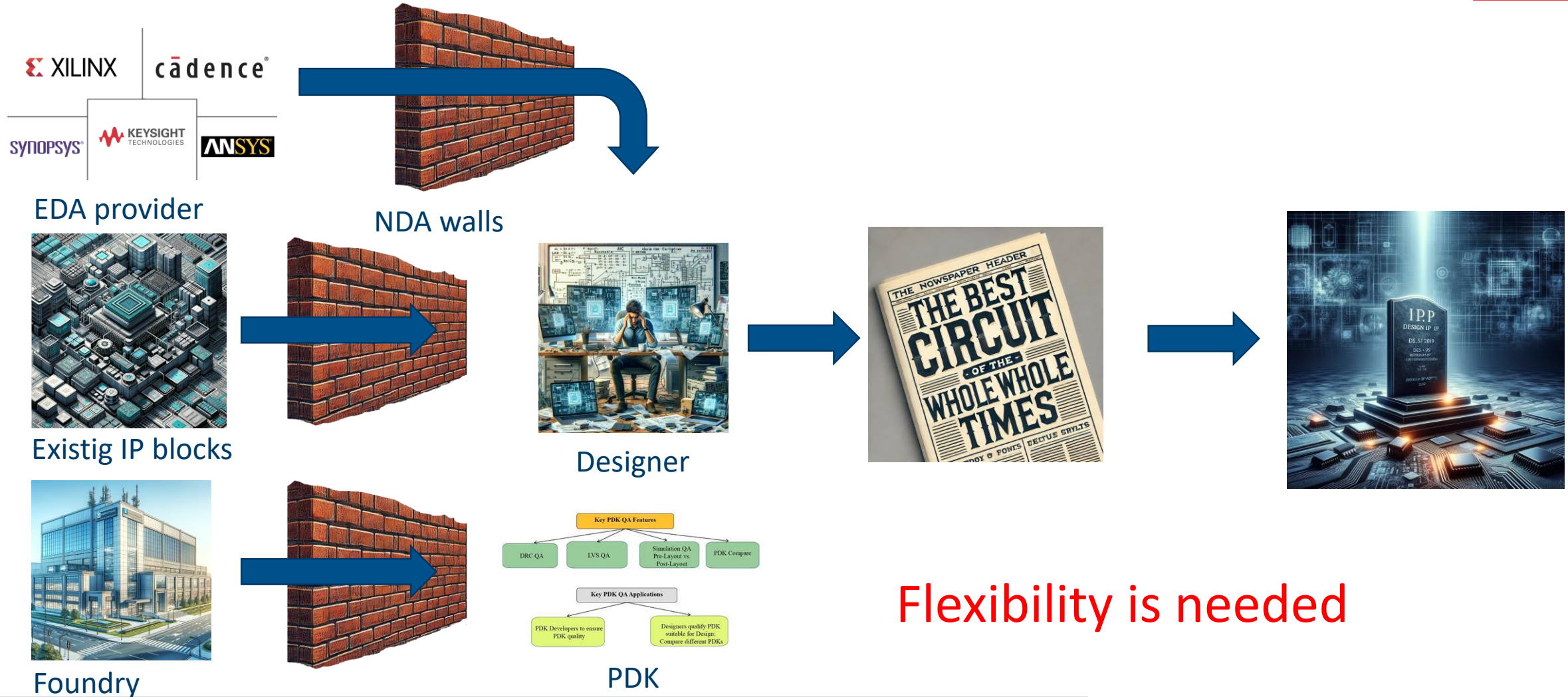
*Cu BEOL from X FAB

- SG13G2 technology was selected for the development of an open source PDK

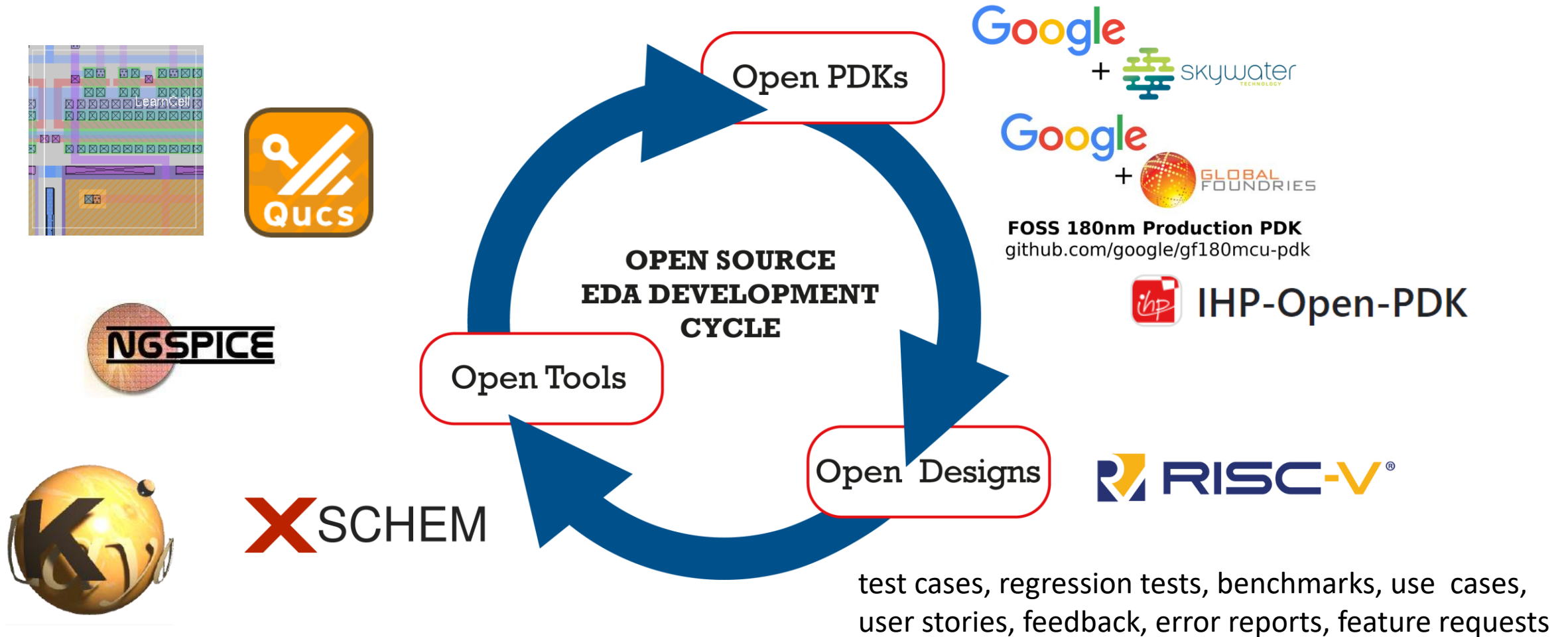
- Target are high-end technology developments, low volume market introduction, technology transfer for potential mass production in commercial fabs
- SG13S & SG13G2 are qualified and ready for Low Volume of high end products
- SG13G3Cu is early access - qualification scheduled 2025



Traditional ASIC development (academic perspective)



Open source ASIC development as an alternative



OpenPDK Project on GitHub

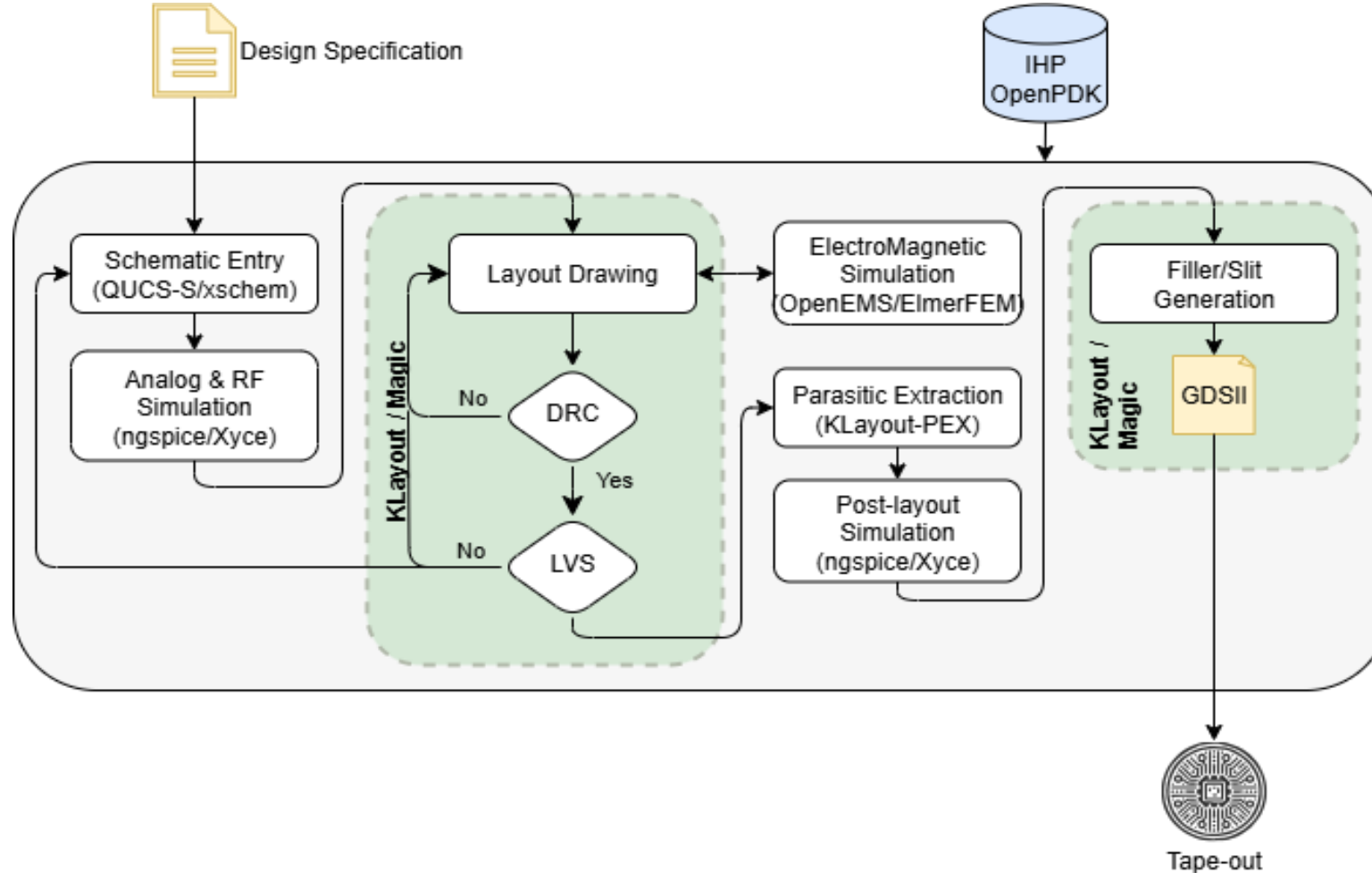


IHP-Open-PDK GitHub repository, <https://github.com/IHP-GmbH/IHP-Open-PDK>, accessed: 2024-07-18.

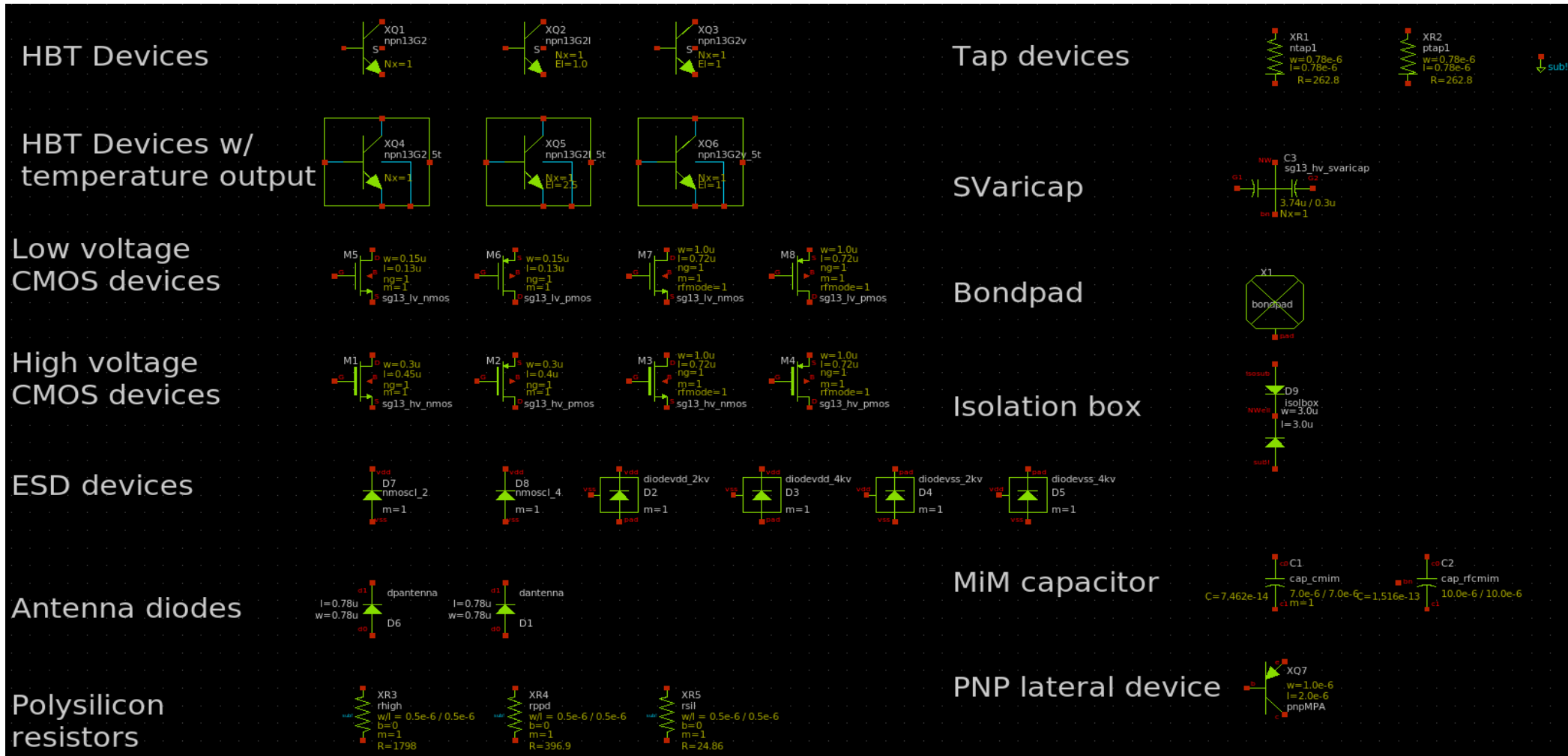
PDK Contents:

- 0 Project Roadmap Gantt chart
- 0 Limited set of standard logic cells (Open130-G2)
- 0 SRAM cells
- 0 IO cells
- 0 Primitive devices (GDS)
- 0 KLayout:
 - 0 Layer property & tech files
 - 0 DRC deck (minimal+maximal rule sets)
 - 0 Filler generation
 - 0 LVS deck
 - 0 Pycells: initial API, 1st priority devices
- 0 ngspice/Xyce simulation models
- 0 xschem:
 - 0 device symbols, settings, testbenches
 - 0 stdcells symbols
- 0 Qucs-S symbols and examples
- 0 OpenEMS: tutorials, scripts, documentation
- 0 SG13G2 Process Specification & Layout Rules
- 0 MOS/HBT/Passive device measurement data
- 0 GDS QA Test structures for DRC/LVS

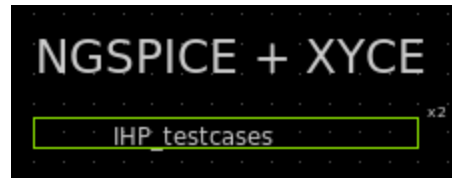
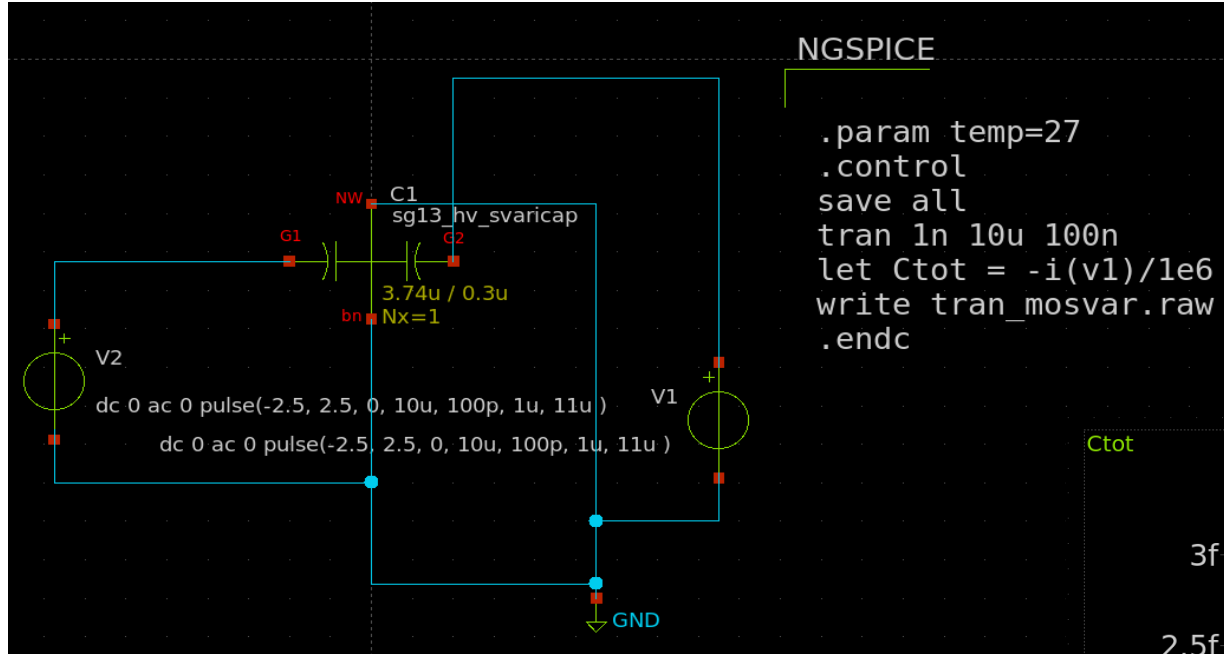
ANALOG/RF OS DESIGN FLOW



XSCHEM: SCHEMATIC DESIGN



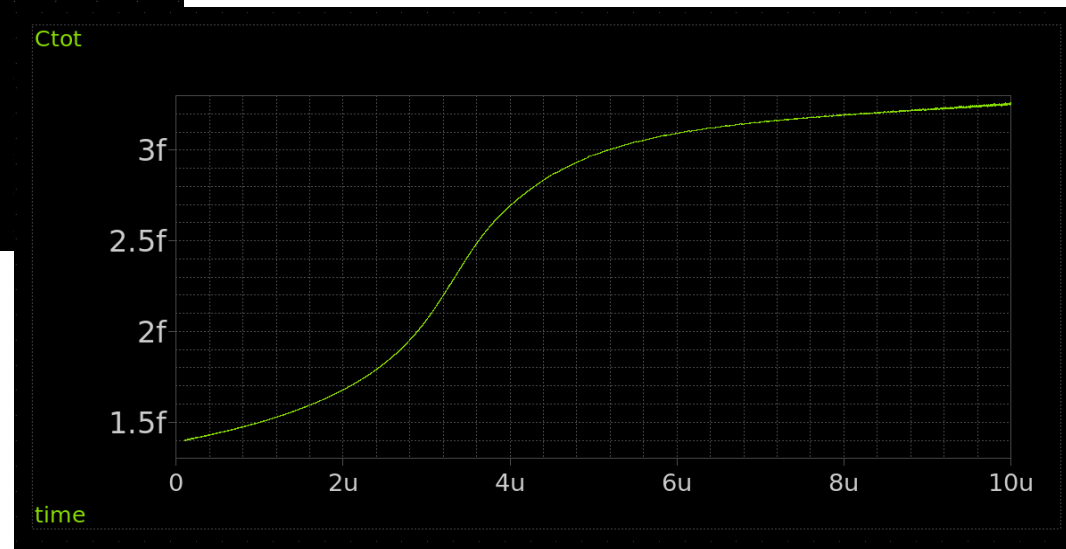
XSCHEM: SIMULATIONS



- Xyce enabled testbenches

Svaricap:

- AC
- TRAN
- S-Parameters



XSCHEM: MC SIMULATIONS



Ctrl-Click to execute launcher

➡ SimulateNGSPICE

➡ OP annotate

.save file can be created with IHP->"Create FET and BIP .save file"

stat ... with staticstial modelling (process) without mismatch!

TT_MODELS

```
** IHP models
.lib cornerMOSlv.lib mos_tt_stat
.lib cornerMOShv.lib mos_tt_stat
.lib cornerHBT.lib hbt_typ_stat
.lib cornerRES.lib res_typ_stat
.lib cornerCAP.lib cap_typ_stat
```

used to run ngspice mc simulation in parallel

```
MC_SIM
SIM=ngspice

.control
set num_threads 1          set num_threads to 1 for small circuits

ac dec 1001 1 100G
let gain_lin = abs(inv_out)
let gain_dB = vdb(inv_out)
meas ac gain_passband_dB max gain_dB
let gain_fc_dB = gain_passband_dB-3
meas ac fc_l when gain_dB = gain_fc_dB
meas ac fc_u when gain_dB = gain_fc_dB cross=last
let GBW = gain_lin[0] * (fc_u-fc_l)

echo results_save_begin
print gain_passband_dB
print fc_l          each printed value will be saved in csv file
print fc_u
print GBW
echo results_save_end

.endc
```

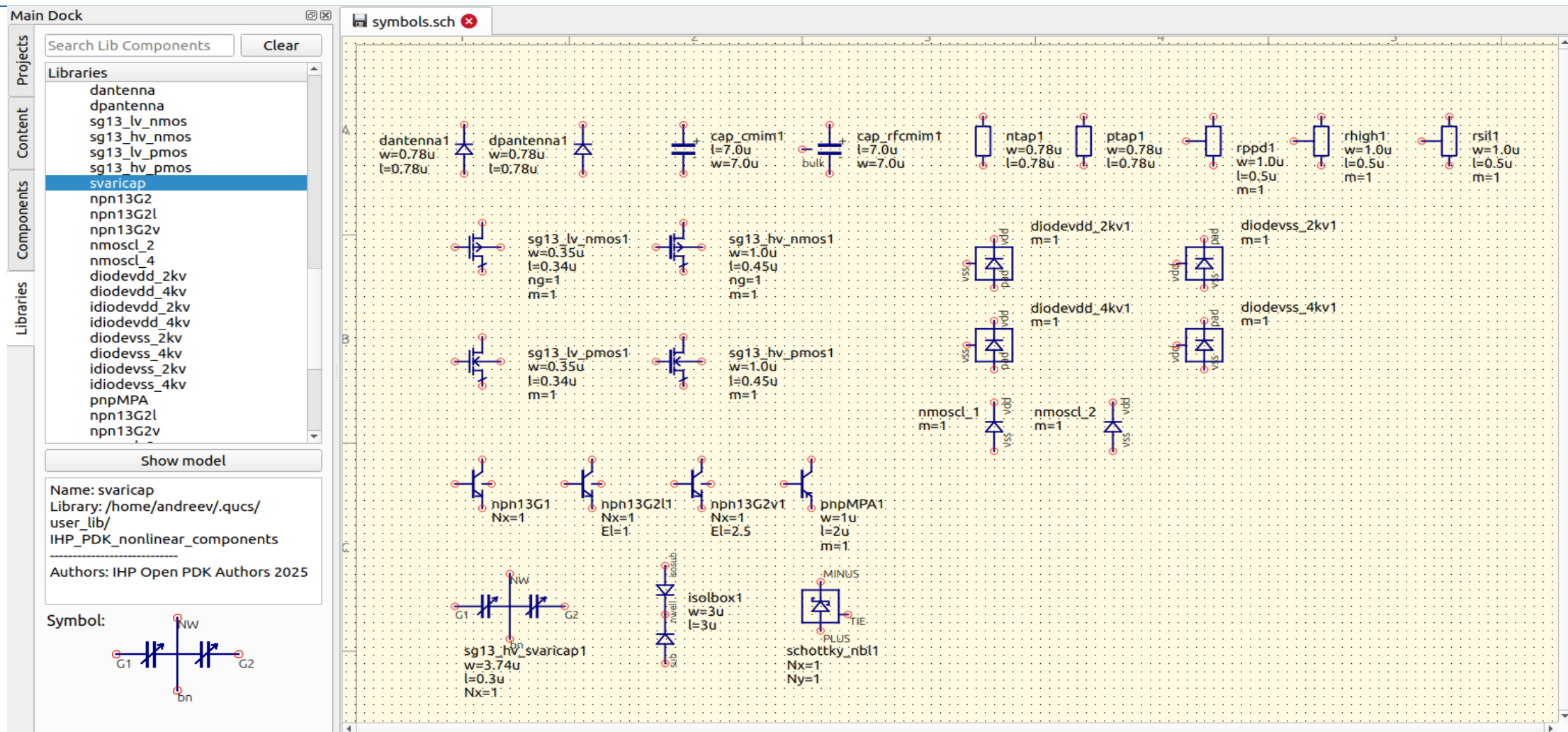
MC_SETTINGS

```
**nr_workers=50
**nr_mc_sims=1000

**results_plot_begin
**gain_passband_dB
**fc_l
**fc_u
**GBW
**results_plot_end
```

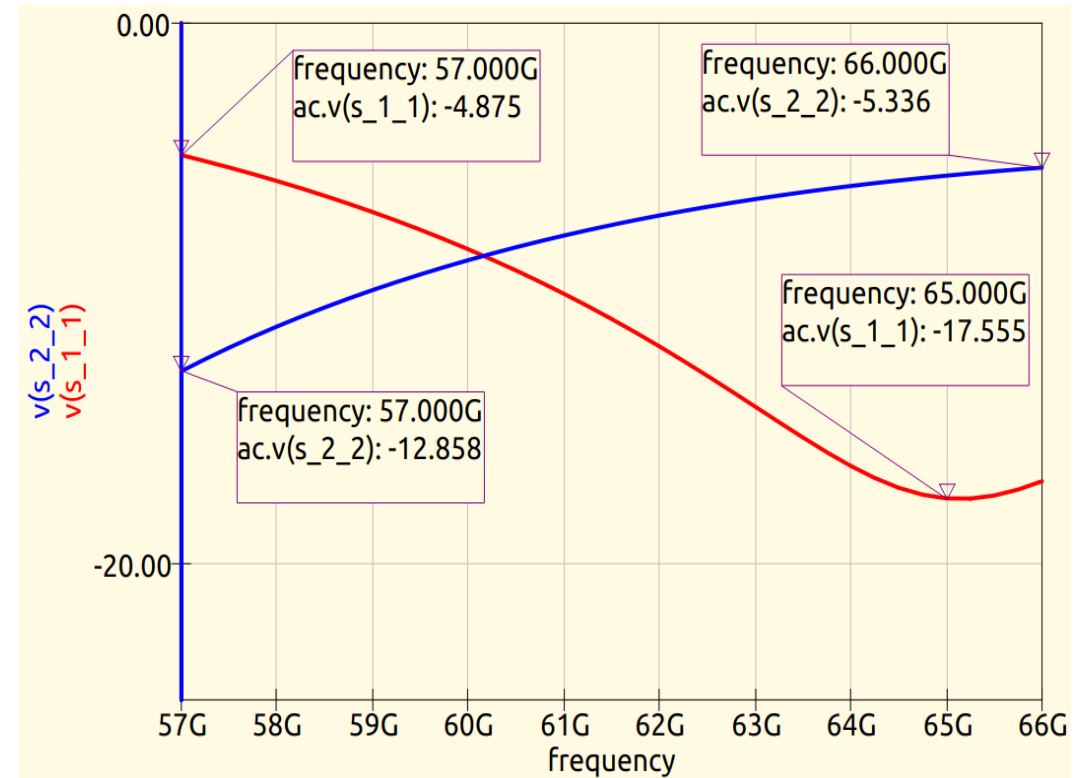
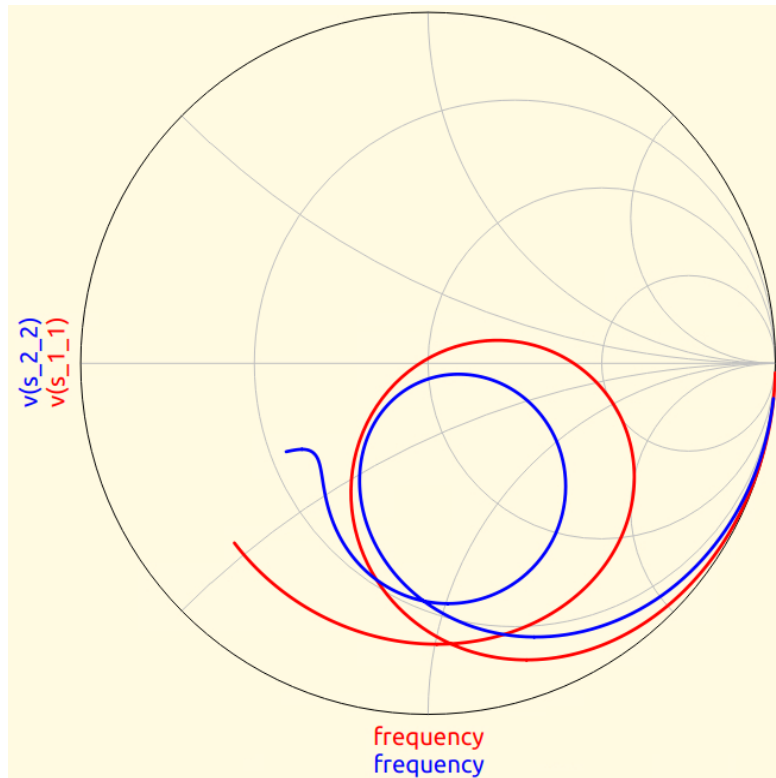
➡ SimulatePARALLEL

QUCS-S: SCHEMATIC DESIGN

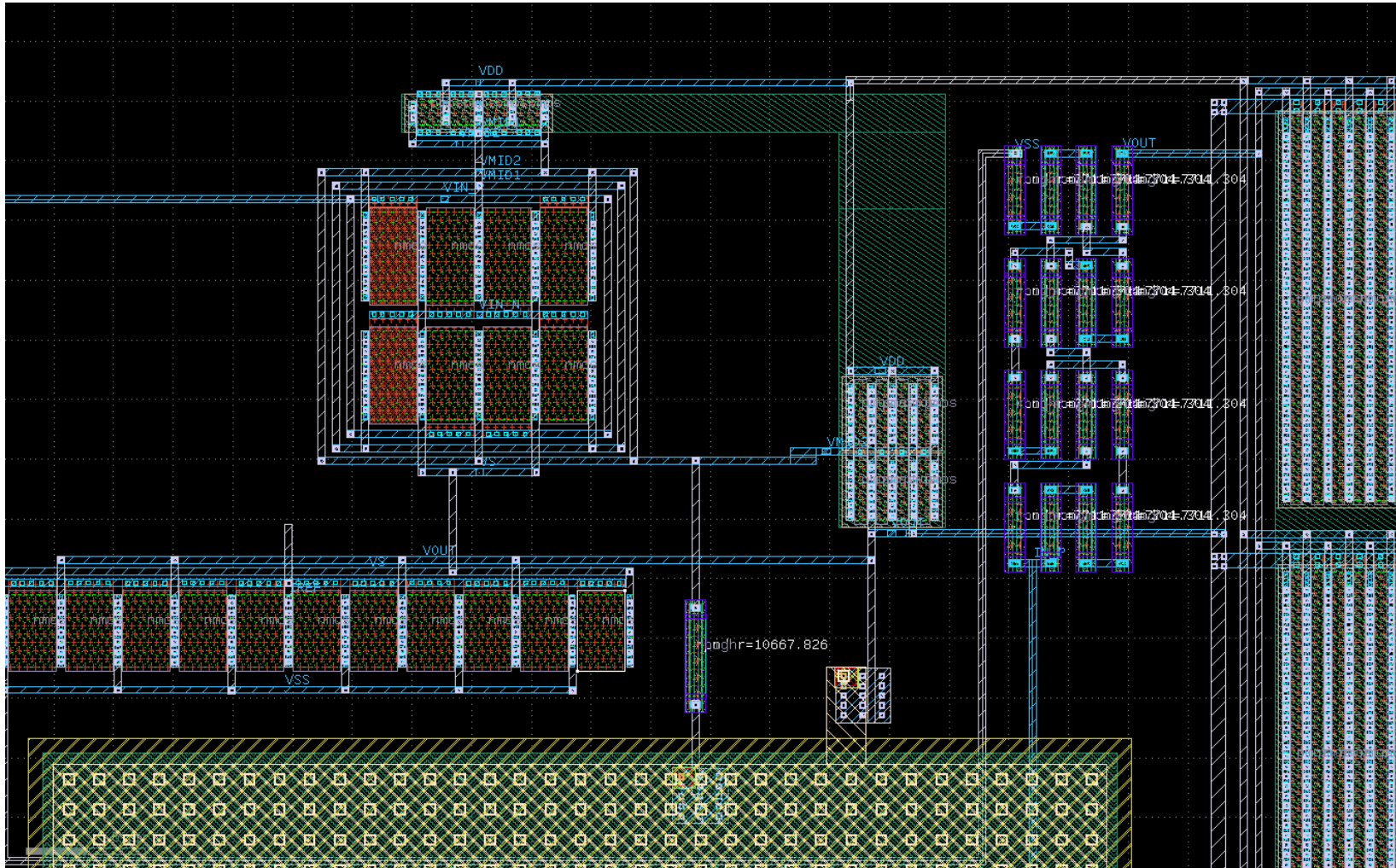




QUCS-S ENVIRONMENT: Viewing Results



KLayout – primary tool for open source layout design



KLayout key features:

- hierarchical view
- parametric cell support
- DRC/LVS checks
- command line batch mode
- XOR and DIFF tools
- Custom scripts in Python/Ruby
- Plugins

<- fragment from LDO design ([link](#))

KLayout example DRC run on `gatpoly` QA cell



KLayout 0.28.12 - sg13g2_qacells.gds [gatpoly]

File Edit View Bookmarks Display Tools Macros Help

Back Forward Select Move Ruler sg13g2

Cells

- gatpoly
- metal1
- nwell

Levels 0 .. 3

Libraries

- ARC
- CIRCLE
- DONUT
- ELLIPSE
- PIE
- ROUND_PATH
- ROUND_POLYGON
- STROKED_BOX
- STROKED_POLYGON
- TEXT

sg13g2_qacells.gds [gatpoly]

Layers

- Activ.drw
- GatPoly.drw
- Cont.drw
- Metal1.drw
- Metal1.pin
- pSD.drw
- NWell.drw
- Substrate.drw
- ThickGateOx.drw
- TEXT.drw

Layer Toolbox

- Color
- Frame color
- Stipple
- Animation
- Style
- Visibility

Marker Database Browser

Database sg13g2.lyrdb File ...

... on layout sg13g2_qacells.gds

Directory

Cell / Category	Count (Not Visited)
By Cell	37 (35)
[gatpoly]	37 (35)
aFil.g	1
aFil.g2	4 (4)
GFil.g	1
Gat.d	8 (8)
M1.j	1 (1)
M1Fil.h	4 (4)
Gat.a	12 (12)
Gat.b	6 (6)
By Category	37 (35)
All	37 (35)

Markers

Info ☒ list shapes

Gat.d [gatpoly]

Min. GatPoly to Activ space = 0.07

Configure Close

KLayout example LVS run on sg13_lv_nmos mosfets



Netlist: LVS
... on layout: sg13_lv_nmos.gds

Netlist | Schematic | Cross Reference | Log

Circuits: sg13_lv_r

Objects	Layout	Reference
sg13_lv_nmos	sg13_lv_nmos	SG13_LV_NMOS
Pins		
Nets		
Devices		
sg13_lv_nn	\$11 / sg13_lv_nmos [L=(N1 / SG13_LV_NMOS [L=0.13, W=0.15]	
sg13_lv_nn	\$12 / sg13_lv_nmos [L=(N2 / SG13_LV_NMOS [L=0.13, W=0.2]	
sg13_lv_nn	\$14 / sg13_lv_nmos [L=(N3 / SG13_LV_NMOS [L=0.15, W=0.2]	
sg13_lv_nn	\$9 / sg13_lv_nmos [L=0. N4 / SG13_LV_NMOS [L=0.15, W=0.3]	
sg13_lv_nn	\$6 / sg13_lv_nmos [L=0. N5 / SG13_LV_NMOS [L=0.3, W=0.3]	
sg13_lv_nn	\$13 / sg13_lv_nmos [L=(N6 / SG13_LV_NMOS [L=0.25, W=0.6]	
S ↔ D	\$35 (1)	D6 (2)
D ↔ S	\$36 (1)	S6 (2)
G	\$37 (1)	G6 (2)
B	\$1 (26)	SUB (27)
sg13_lv_nn	\$16 / sg13_lv_nmos [L=(N7 / SG13_LV_NMOS [L=0.15, W=0.6]	
sg13_lv_nn	\$4 / sg13_lv_nmos [L=3. _PATTERN_37 / SG13_LV_NMOS [L=3.74, W=5.55]	
sg13_lv_nn	\$5 / sg13_lv_nmos [L=4. _PATTERN_40 / SG13_LV_NMOS [L=4.6, W=7.09]	

Klayout LVS ruledeck:

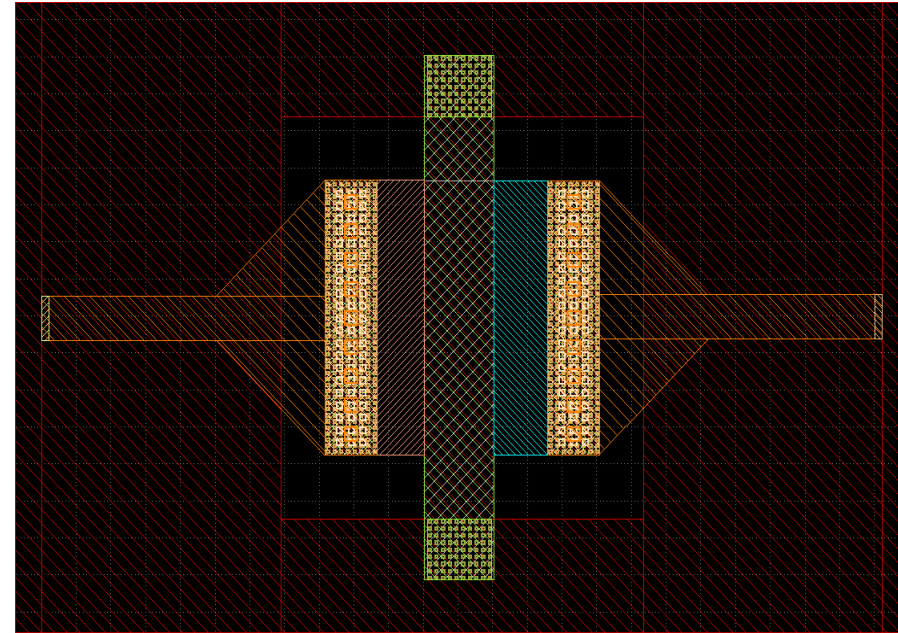
- accepts CDL netlist,
- extracts devices from layout,
- creates extracted netlist,
- performs checks and compares:
 - Pins
 - Nets
 - Devices
- reports inconsistencies
- can run in batch mode

- Why OpenEMS?
 - Cross-platform: Linux, Windows, macOS
 - Easy to compile and set up
 - Validated against commercial EM solvers



openEMS is a free and open electromagnetic field solver using the FDTD method.

- Down-sides?
 - Hardcoded structures limit high level abstraction
 - CPU-based kernel results in slower simulation times

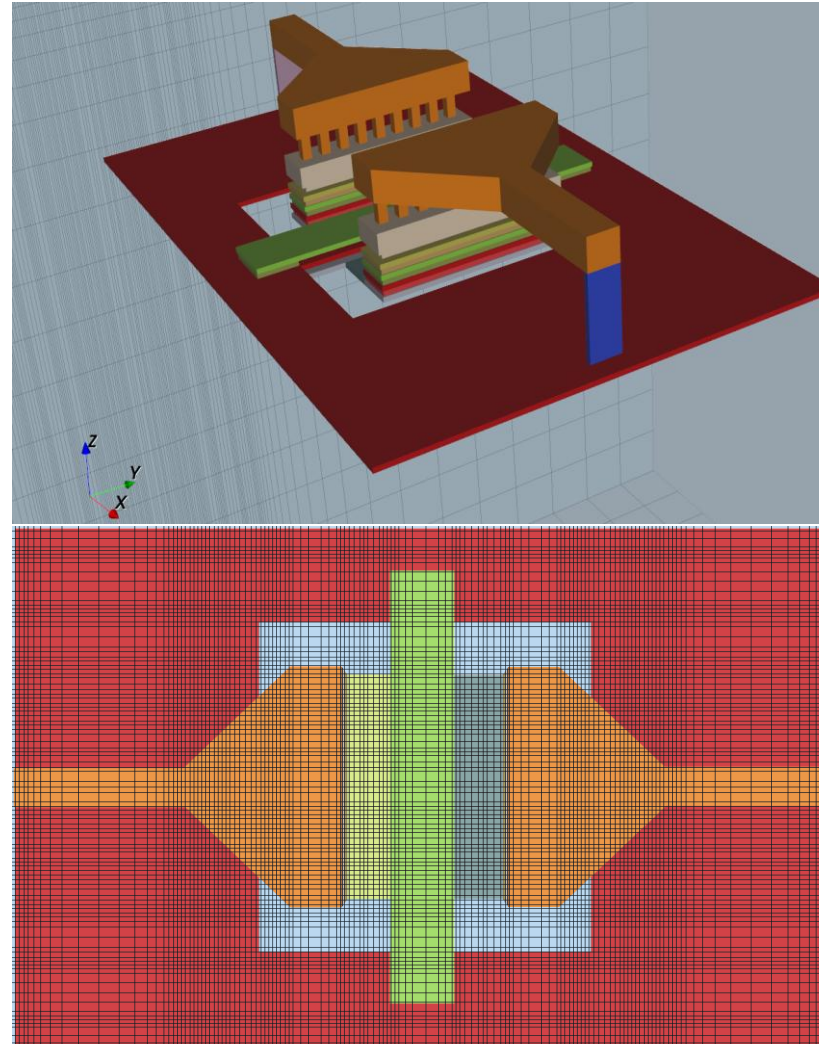


np.array("good luck")

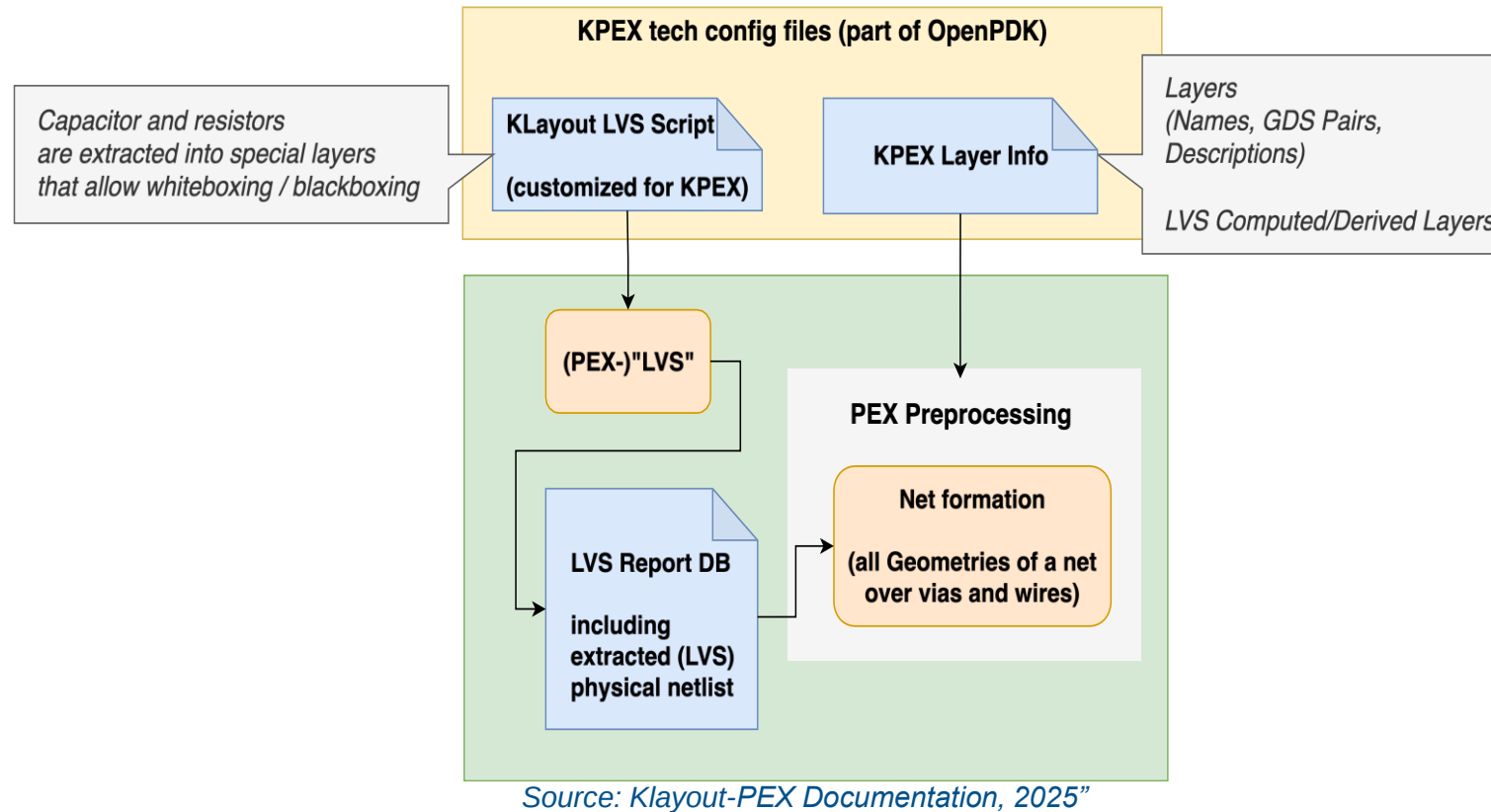
OPENEMS – AN OPEN EM SOLVER



- Better Streamlining
 - Python/Matlab based
 - Read geometries directly from GDSII
 - Read technology stackup from XML file



KLAYOUT PARASITIC EXTRACTION

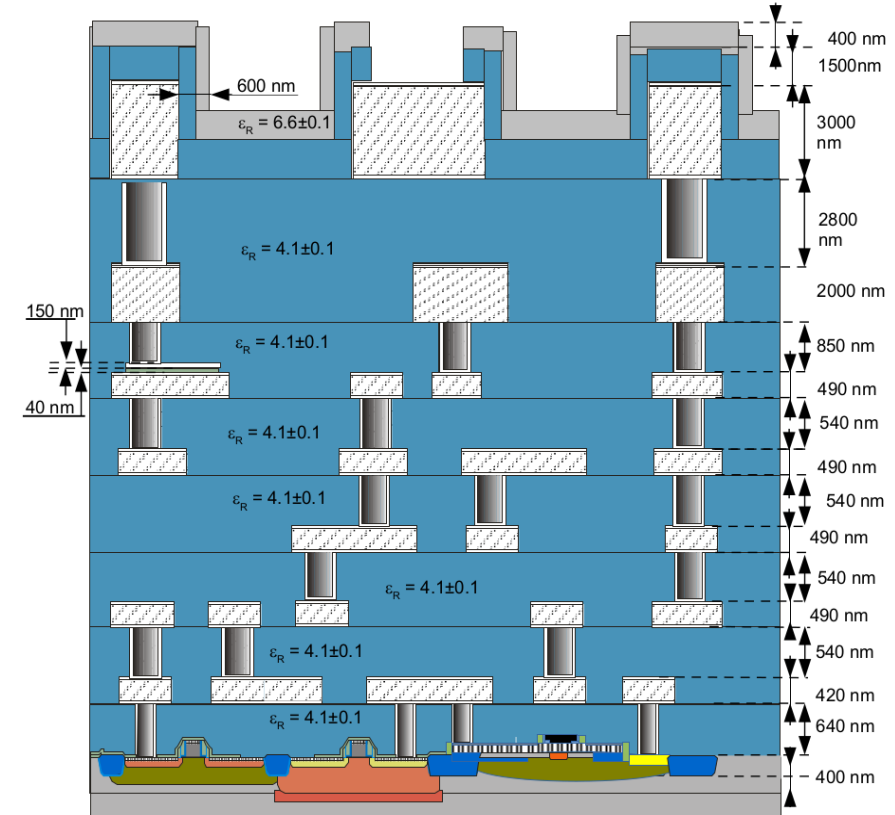


KLAYOUT PARASITIC EXTRACTION



- Process Information

- All information is already incorporated into the repo
- Available processes: IHP sg13g2, Sky130A

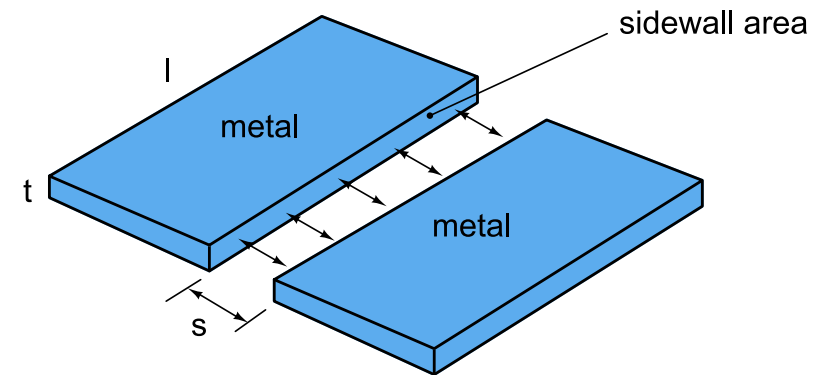
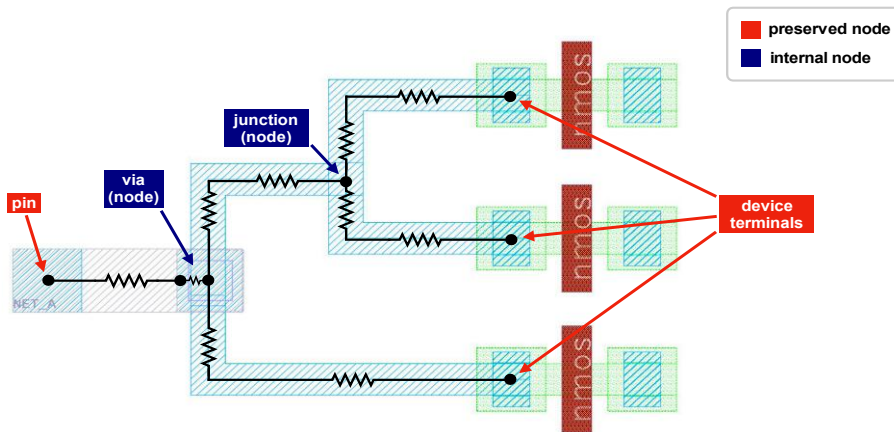
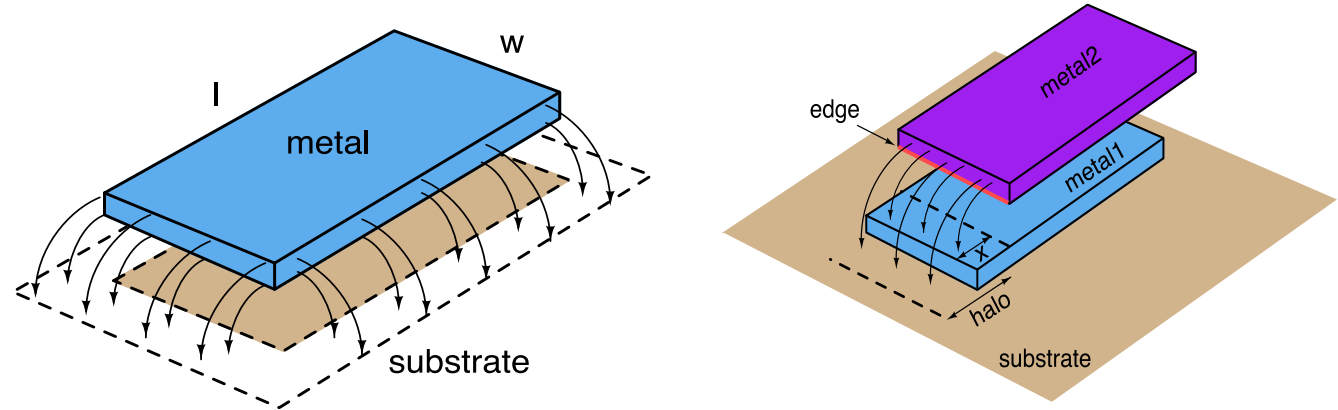


Source: IHP Open PDK Documentation, 2024"

KLAYOUT PARASITIC EXTRACTION

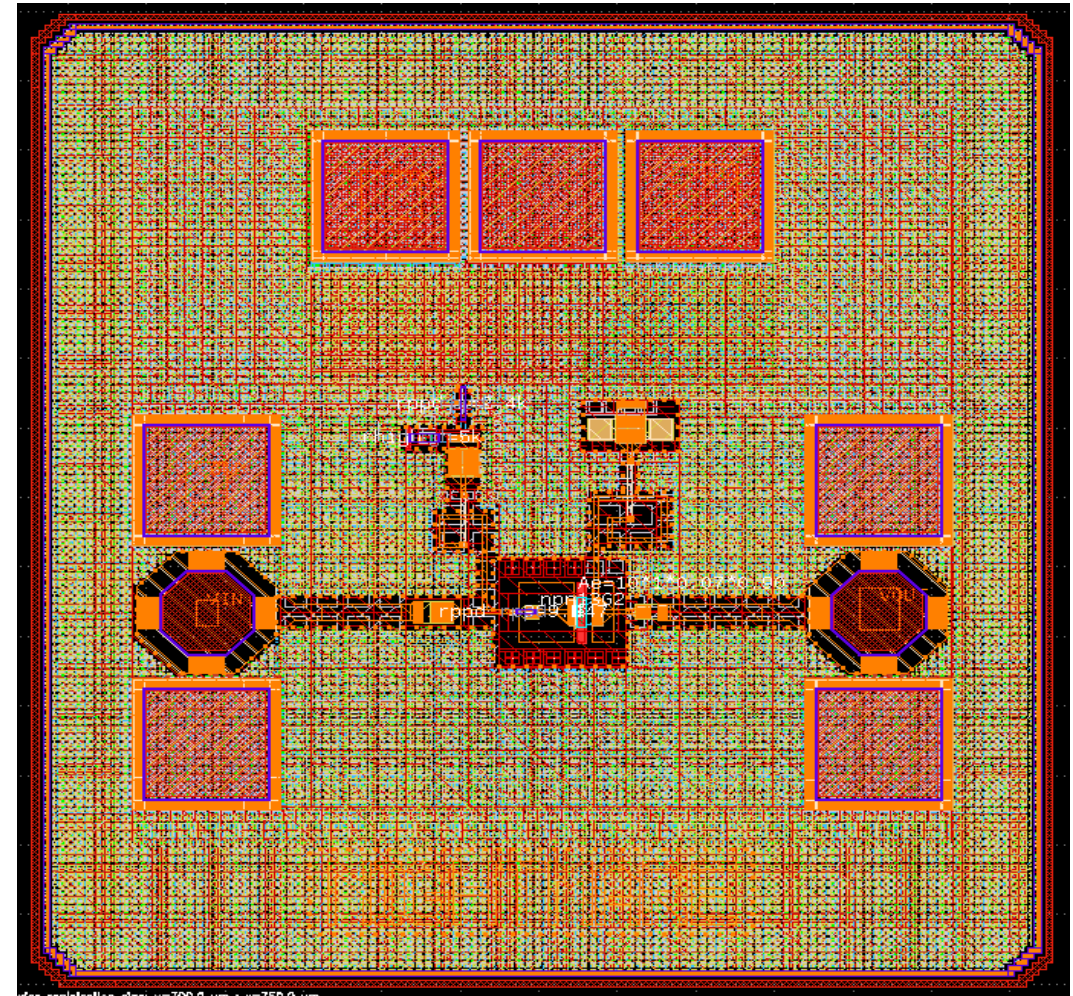
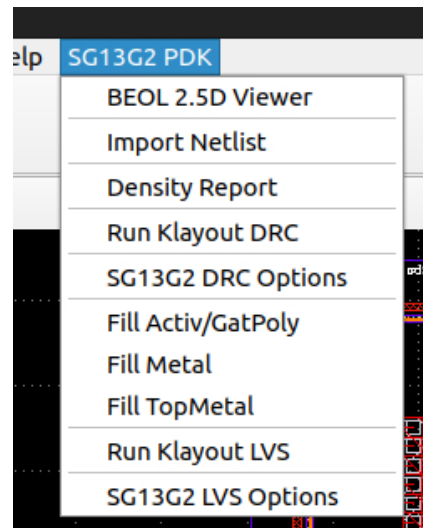
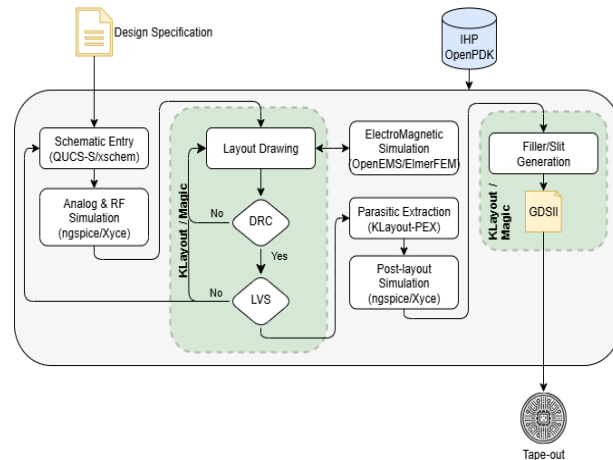


- Extraction modes
 - Configurable engines (magic engine currently supported)
 - CC, RC, R, L, RCC



Source: Klayout-PEX Documentation, 2025"

FILLER GENERATION



How to reach us



Email

→ openpdk@ihp-microelectronics.com

GitHub

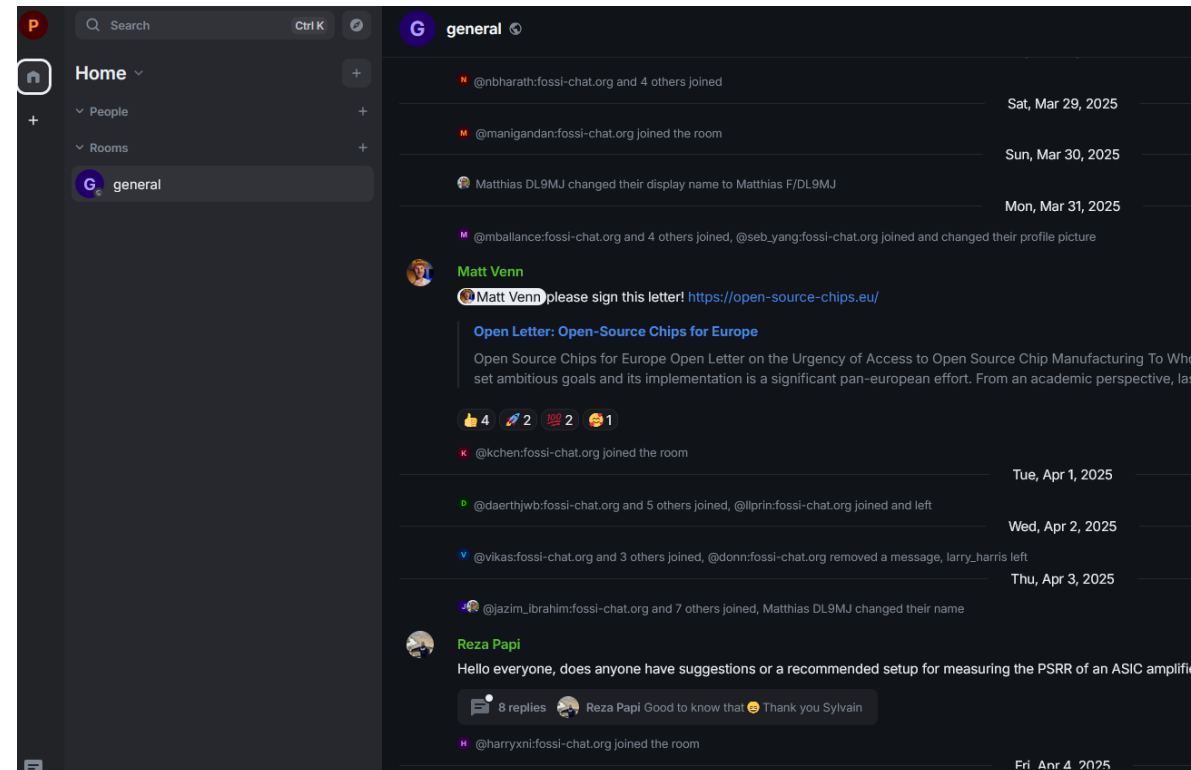
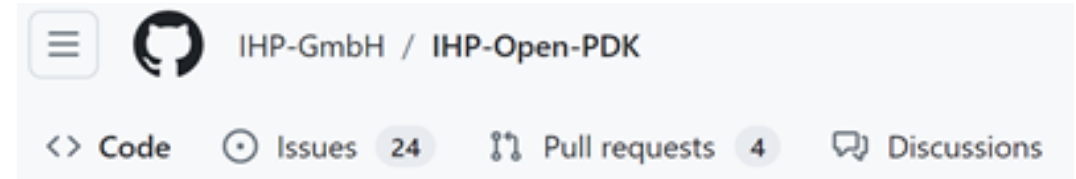
→ issues – for reporting issues, questions

→ discussions – for requesting features

Fossi-Chat

→ general discussions

→ announcements





Leibniz Institute
for high
performance
microelectronics

Laying the Groundwork: A Soft Start to Build a Strong Foundation

Repository reference:
modules/modules_0_foundations

Quick Note on the setup



- 0 For some exercises, you will need Python for post-processing or specific operations. A virtual environment is already set up; enter it with **enter_venv** command
- 0 The repository for the PDK can be found at: <https://github.com/IHP-GmbH/IHP-Open-PDK>
- 0 The repository for this course can be found at: <https://github.com/IHP-GmbH/IHP-AnalogAcademy>

- The image displays the IHP-Open-PDK design kit interface, showing a list of test cases organized into five categories: DC, Transient, AC, Monte Carlo, and S-param. Each category contains a list of test cases, each represented by a green bar with a label and a number.

IHP-Open-PDK

DC

 - dc_iv_rmos
 - dc_iv_rmos
 - dc_iv_rmos
 - dc_iv_rmos
 - dc_mos_temp
 - dc_mos_cs_temp
 - dc_rcs_temp
 - dc_ntap
 - dc_ptap1
 - dc_diode_op
 - dc_diode_temp
 - dc_hbt_13q2
 - dc_pripMFA
 - dc_logic_not

Transient

 - tran_mim_cap
 - tran_logic_not
 - tran_logic_nand

AC

 - ac_iv_rmosif
 - ac_mim_cap

Monte Carlo

 - mc_iv_rmos_cs_loop
 - mc_iv_rmos_cs_loop
 - mc_iv_rmos_cs_loop
 - mc_iv_rmos_cs_loop
 - mc_res_op
 - mc_hbt_13q2
 - mc_hbt_13q2_ac
 - mc_mim_xap_ac

S-param

 - sp_mim_cap
 - sp_parasitic_cap
 - sp_rfmm_cap

XSCHEM

Copyright 2023 IHP PDK Authors
sg13q2_tests/IHP_testcases.sch

2024-10-15 12:52:12



Ngspice Code Block: Defines the simulation setup



- 0 **.controlendc**: Specifies a sequence of commands to control the simulation
- 0 **Save all**: Saves all operating points, node voltages, currents, etc., for later review in the Xschem terminal
- 0 **op**: Executes a DC operating point analysis
- 0 **The dc Vds**: Performs a DC parametric sweep of **Vds** from 0V to 1.2V in 0.01V steps (and similarly for other parameters)
- 0 **Write**: Saves the final DC results to a file, `dc_lv_nmos.raw`, in the simulation directory (auto generated folder)

NGSPICE

```
.param temp=27
.control
save all
op
dc Vds 0 1.2 0.01 Vgs 0.3 0.5 0.05
write dc_lv_nmos.raw
.endc
```

Additional Setup



-0 **Model:** Specifies which model we want to use. In this case we are using the low voltage transistor in the typical-typical corner

```
MODEL
.lib cornerMOSlv.lib mos_tt
```

-0 **Waveform Reload Launcher:** This is where we define the path to the data that we want to plot in the diagram in the schematic, more on this later..

➡ load waves Ctrl + left click

```
name=h5
descr="load waves Ctrl + left click"
tclcommand="xschem raw_read $netlist_dir/dc_lv_nmos.raw dc"
```

Now Its Your Turn (Creating the first schematic)

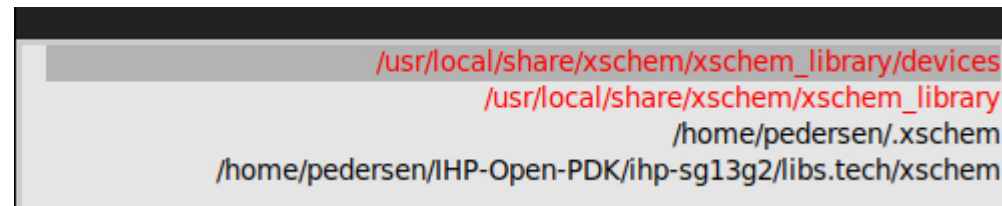


Stay Organized! Before we begin, we need to create a **.sch** file for our schematic. Navigate to your desired location and create the schematic by running:

```
touch test.sch
```

Open Schematic: Open the schematic, then navigate to **Components**  (Ctrl + i)

Here, you'll find both the **Xschem library** and the **Open PDK library**.



← Xschem Lib

← Open PDK

Placing Components

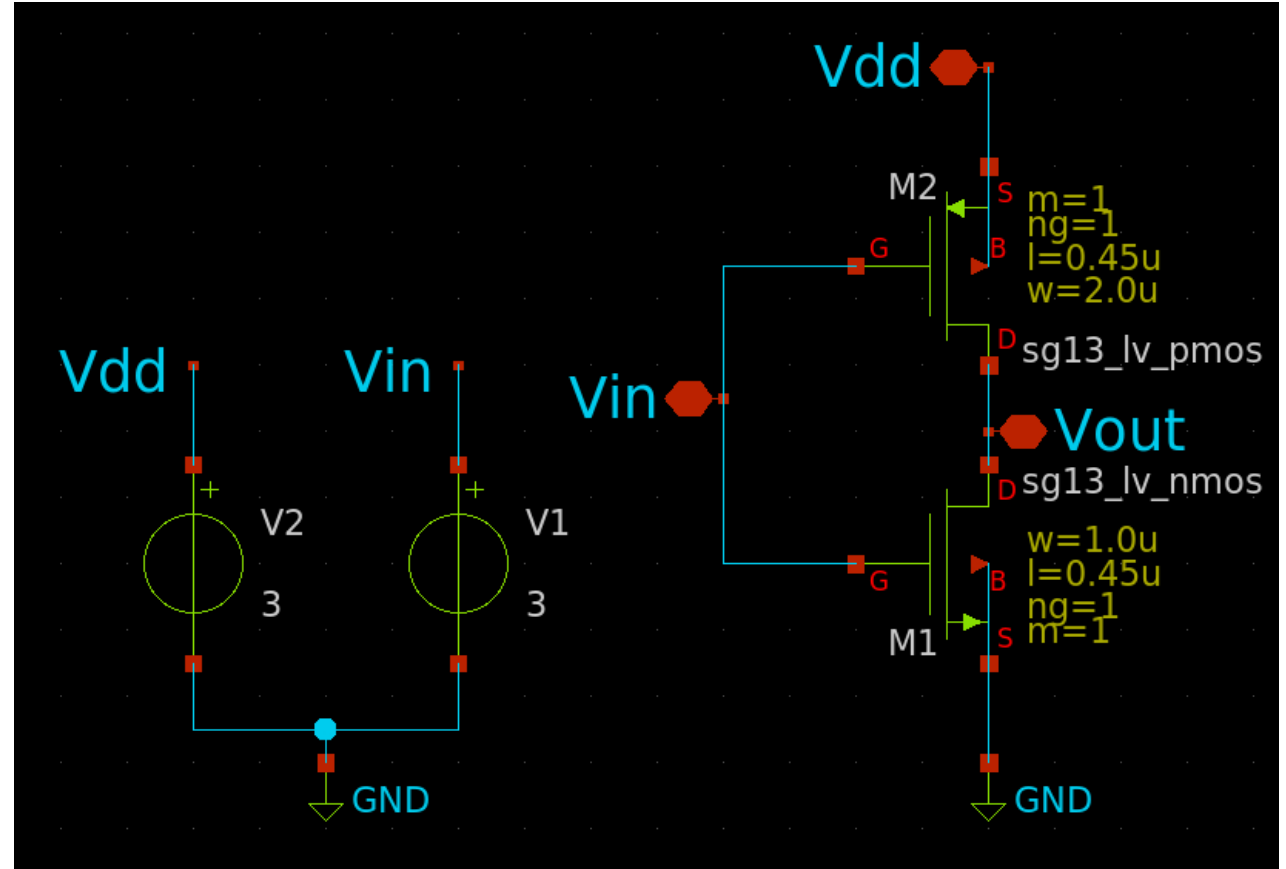


List of components:

- 0 xschem_library/devices: vsource.sym (x2)
- 0 xschem_library/devices: gnd.sym (x2)
- 0 xschem_library/devices: lab.sym (x2)
- 0 xschem_library/devices: iopin.sym (x3)
- 0 libs.tech/xschem/sg13g2_pr: sg13_lv_nmos
- 0 libs.tech/xschem/sg13g2_pr: sg13_lv_pmos

Remember to double the width of the PMOS!

- 0 To wire components press **W**
- 0 To center the schematic press **F**
- 0 To rotate press (**shift + R**) to flip press (**shift + F**)



Viewing The Netlist



Being comfortable with netlist viewing is essential, as it reveals how the simulator interprets your circuit.

To extract the netlist:

- 0 Open **Options** and navigate to **Netlist Format / Symbol Mode**.
- 0 Select **SPICE Netlist** to ensure compatibility with Ngspice.
- 0 Go to the **Simulation** tab and click **Show Netlist After Netlist Command (Shift + A)**.

```
** sch_path: /home/pedersen/projects/IHP-AnalogAcademy/module
**.subckt inverter Vout Vdd Vin
*.iopin Vout
*.iopin Vdd
*.iopin Vin
V1 Vin GND 3
V2 Vdd GND 3
XM1 Vout Vin GND GND sg13_lv_nmos w=1.0u l=0.45u ng=1 m=1
XM2 Vout Vin Vdd Vdd sg13_lv_pmos w=2.0u l=0.45u ng=1 m=1
**.ends
.GLOBAL GND
.end
```

Viewing The Netlist



-0 First Section indicates the subcircuit which is commented out in this case since we are looking at a standalone Schematic

```
** .subckt inverter Vout Vdd Vin  
* .iopin Vout  
* .iopin Vdd  
* .iopin Vin
```

-0 Next section defines the voltage sources in this Vin is set to 3V relative to **Gnd** and same for **Vdd**

```
V1 Vin GND 3  
V2 Vdd GND 3
```

-0 **Transistor definition:** M<XName> <Drain> <Gate> <Source>
<Bulk> <Model> W=<Width> L=<Length> ng=<number of gates>
m=<multiplier>

```
XM1 Vout Vin GND GND sg13_lv_nmos w=1.0u l=0.45u ng=1 m=1  
XM2 Vout Vin Vdd Vdd sg13_lv_pmos w=2.0u l=0.45u ng=1 m=1
```

-0 **.Global GND:** As the name indicates it's a global node reference

```
.GLOBAL GND
```

Complete the Testbench Using These Hints



4.1.1 Pulse

General form:

PULSE(V1 V2 TD TR TF PW PER NP)

11.3.10 .TRAN: Transient Analysis

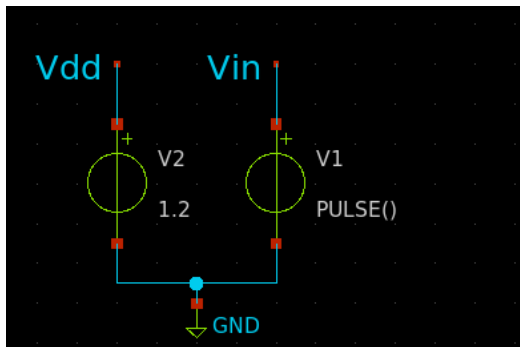
General form:

.tran tstep tstop <tstart <tmax>> <uic>

-0 Xschem_library/Devices: code_shown.sym (x2)

(Hint: Check examples in the PDK by running *xschem* in the terminal)

-0 Vdd/Logic High: 1.2V



```
MODEL
.lib
```

```
NGSPICE
.control
save all
write test_inverter.raw
.endc
```

Something is missing!



```
.lib cornerMOSlv.lib mos tt
```



```
.control
save all
tran 50n 2u
write test_inverter.raw
.endc
```



A circuit diagram showing a voltage source V_1 in series with a resistor R_1 . The voltage source is represented by a circle with a '+' sign at the top. The resistor R_1 is represented by a rectangle. The load resistor R_L is represented by a circle. The current i is indicated by an arrow pointing downwards through the load resistor.

```
PULSE(0 1.2 0.5u 10n 10n 1u 2u 1)
```

Final Schematic



Commands in Xschem Terminal:

- o show all (shows for example transistor small signal parameters)
- o display (shows available data)
- o Plot (plots data 😊 try with plot vout)

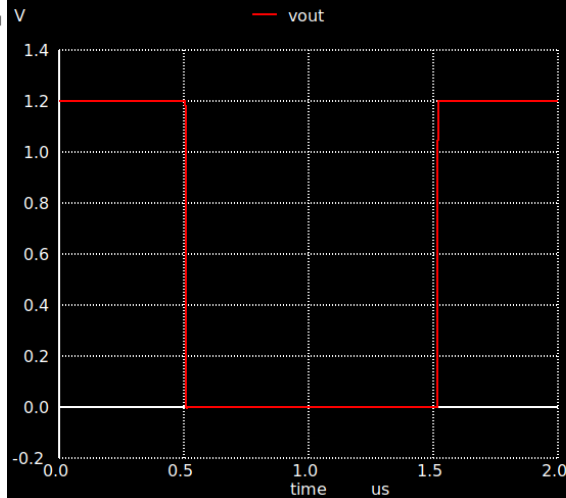
Doing analysis at TEMP = 27.000000 and TNOM = 27.000000

Using SPARSE 1.3 as Direct Linear Solver

Initial Transient Solution

Node	Voltage
vin	0
vdd	1.2
vout	1.2
v2#branch	-3.09018e-10
v1#branch	8.18116e-11

No. of Data Rows : 80
binary raw file "test_inverter.raw"
ngspice 74 -> █

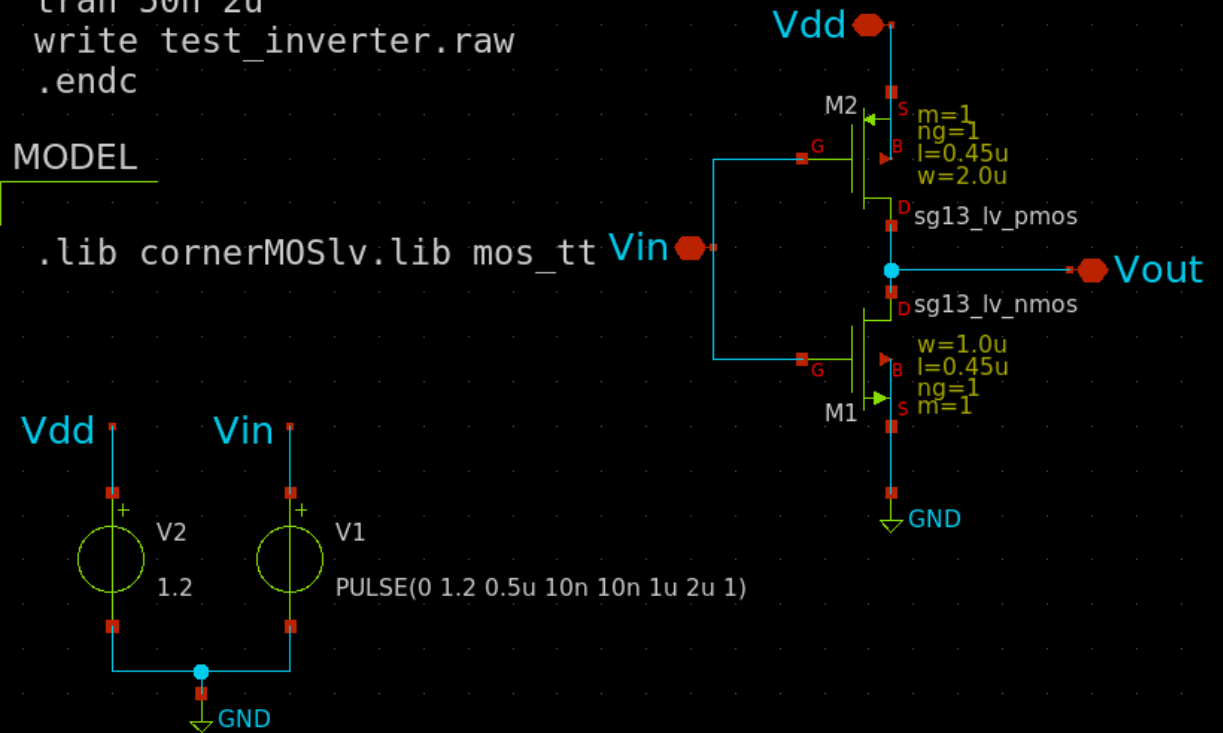


NGSPICE

```
.control
save all
tran 50n 2u
write test_inverter.raw
.endc
```

MODEL

```
.lib cornerMOSlv.lib mos_tt
```



Setting A Fixed Plot In Schematic

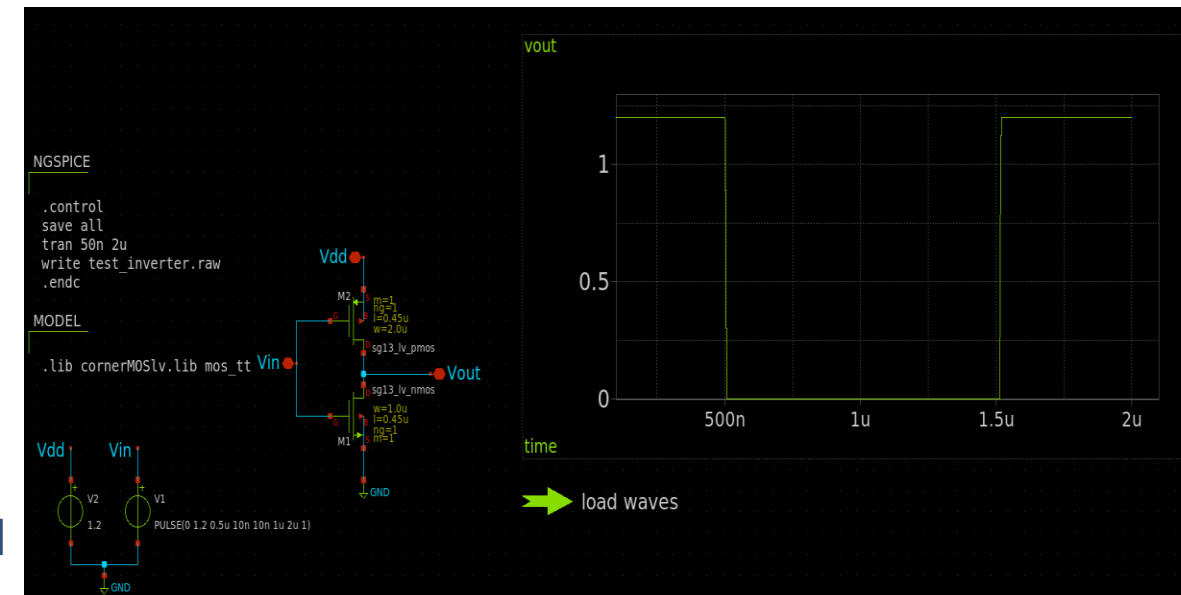


Lets review the simulation outcome:

- 0 Navigate to the directory of the schematic. You should see a simulation folder.
- 0 Inside this folder you should find the **.raw** file and the **.spice** file. We will use the **.raw** file to set the plot since this contains our simulation data

Next insert the plot in the schematic:

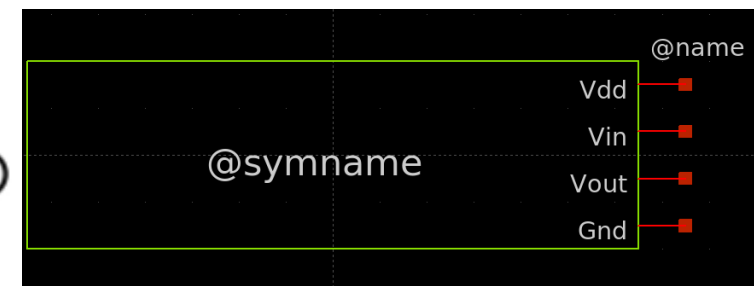
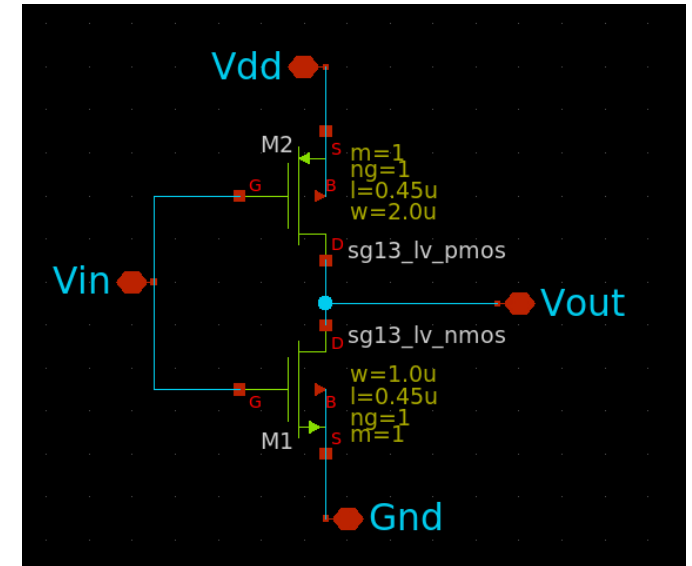
- 0 Navigate to the **Simulation** tab and under **graphs** press **add waveform graph** and **add waveform reload launcher**
- 0 Double click the launcher and make sure you point to right **.raw** file
- 0 After this load the waves (Ctrl + Left click) and double click on the wave form viewer. Now select the data to be plotted
- 0 By pressing **F** on both axes you can scale the plot



Creating a Symbol



- 0 The simplest way to create a symbol (from the testbench) is by duplicating the existing schematic and renaming the copy to **inverter_tb.sch**
The original schematic can then be used to define the symbol subcircuit
- 0 Once this is done, we can delete everything except for the inverter (Remember to put IO pin on the source of the NMOS)
- 0 For Creating the Symbol press **A**
- 0 This creates a symbol file in the directory of the schematic, navigate to this file and open it by writing (**xschem inverter.sym**)
- 0 Now you can drag the already existing lines to create the boundaries or use the tools seen in the toolbar to define custom shapes



Inserting Symbol In Schematic

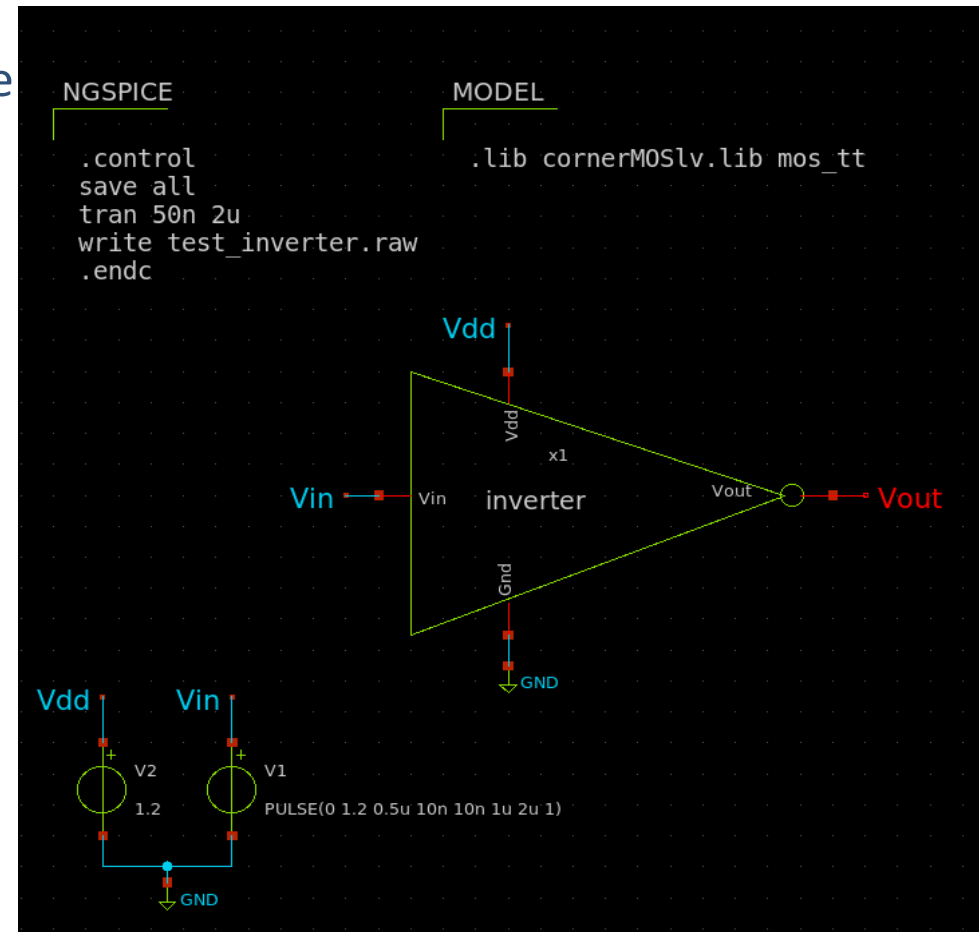


- 0 When the symbol is done remember to save and navigate over to the testbench file.
- 0 Now open component library and press **current dir**
- 0 Select your symbol and insert it in the testbench and connect accordingly and review the netlist. After this run the simulation!

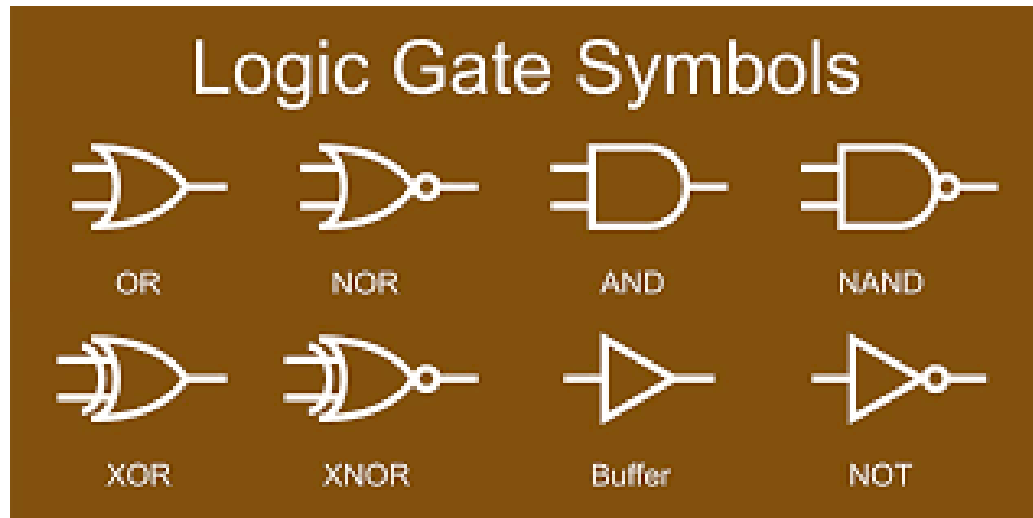
```
.subckt inverter Vdd Vin Vout Gnd
*.iopin Vout
*.iopin Vdd
*.iopin Vin
*.iopin Gnd
XM1 Vout Vin Gnd Gnd sg13_lv_nmos w=1.0u l=0.45u ng=1 m=1
XM2 Vout Vin Vdd Vdd sg13_lv_pmos w=2.0u l=0.45u ng=1 m=1
.ends
```

- 0 Note that in the netlist the subcircuit definition is uncommented

Current dir



- 0 Try and Implement some of the standard logic gates





Leibniz Institute
for high
performance
microelectronics

Catching Up / Lunch Time !

A decorative graphic on the left side of the slide consisting of various colored lines (red, blue, grey) and small circles, resembling a circuit board or a network diagram.

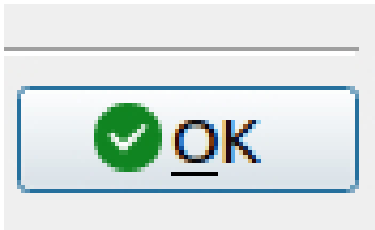
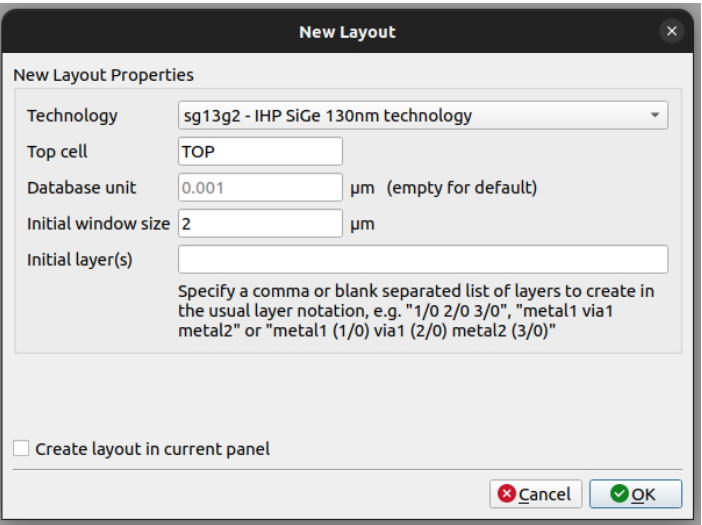
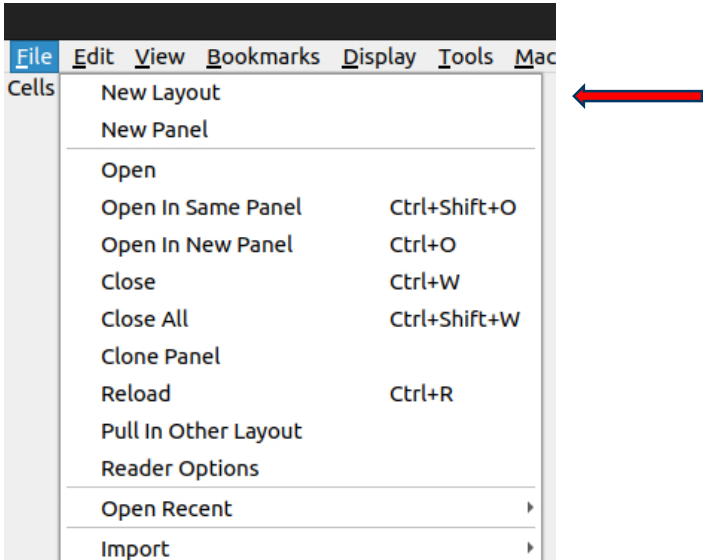
Next session:
Introduction to KLayout



Leibniz Institute
for high
performance
microelectronics

Getting Familiar With KLayout

Getting Started!

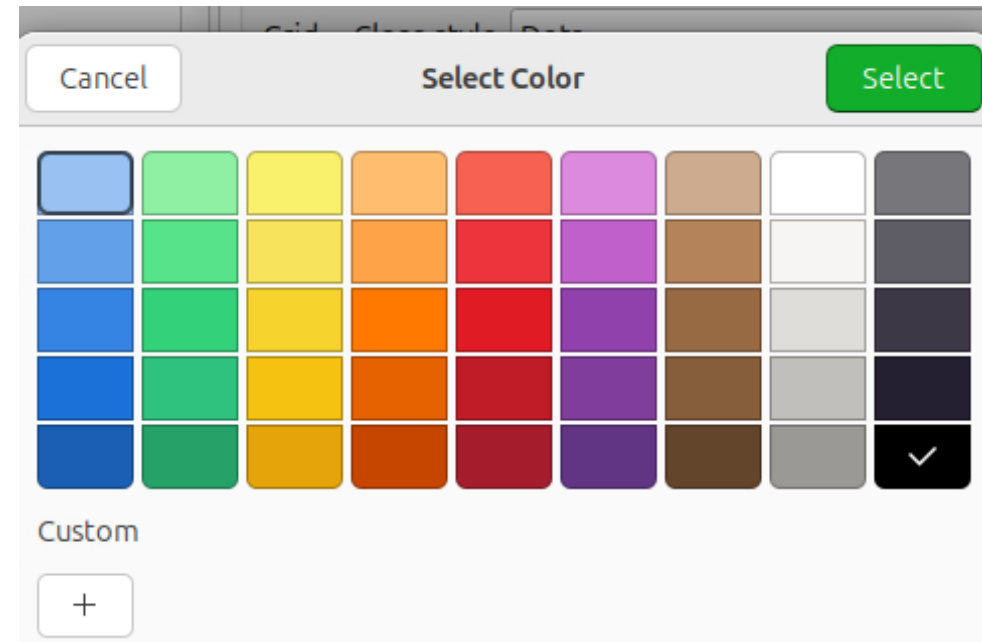
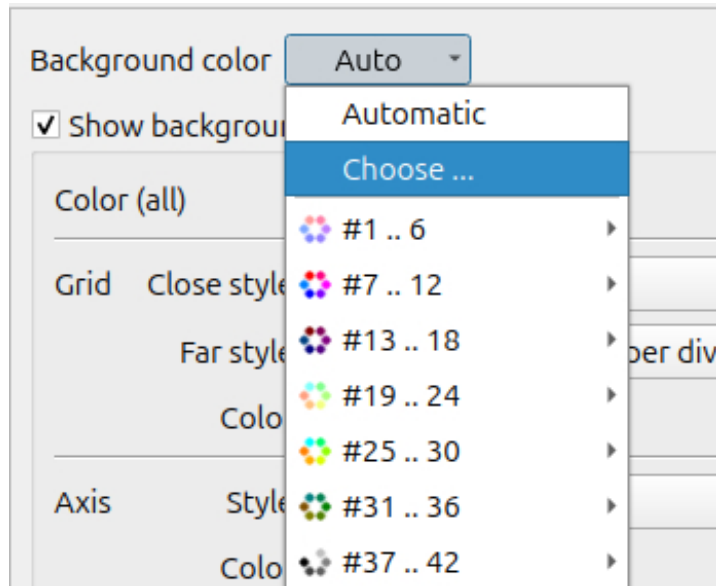


Launching Klayout: `klayout -e` (edit mode)

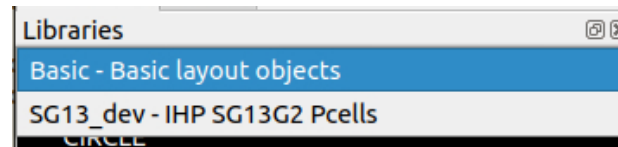
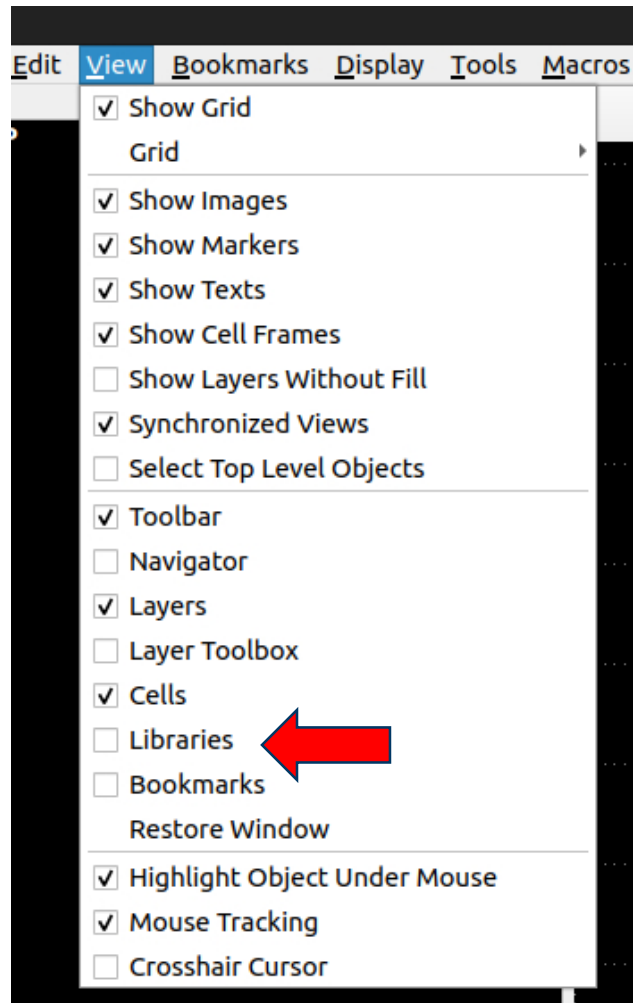
Pro Tip!



To set dark mode navigate to File -> Setup -> Display -> Background



Viewing The Devices



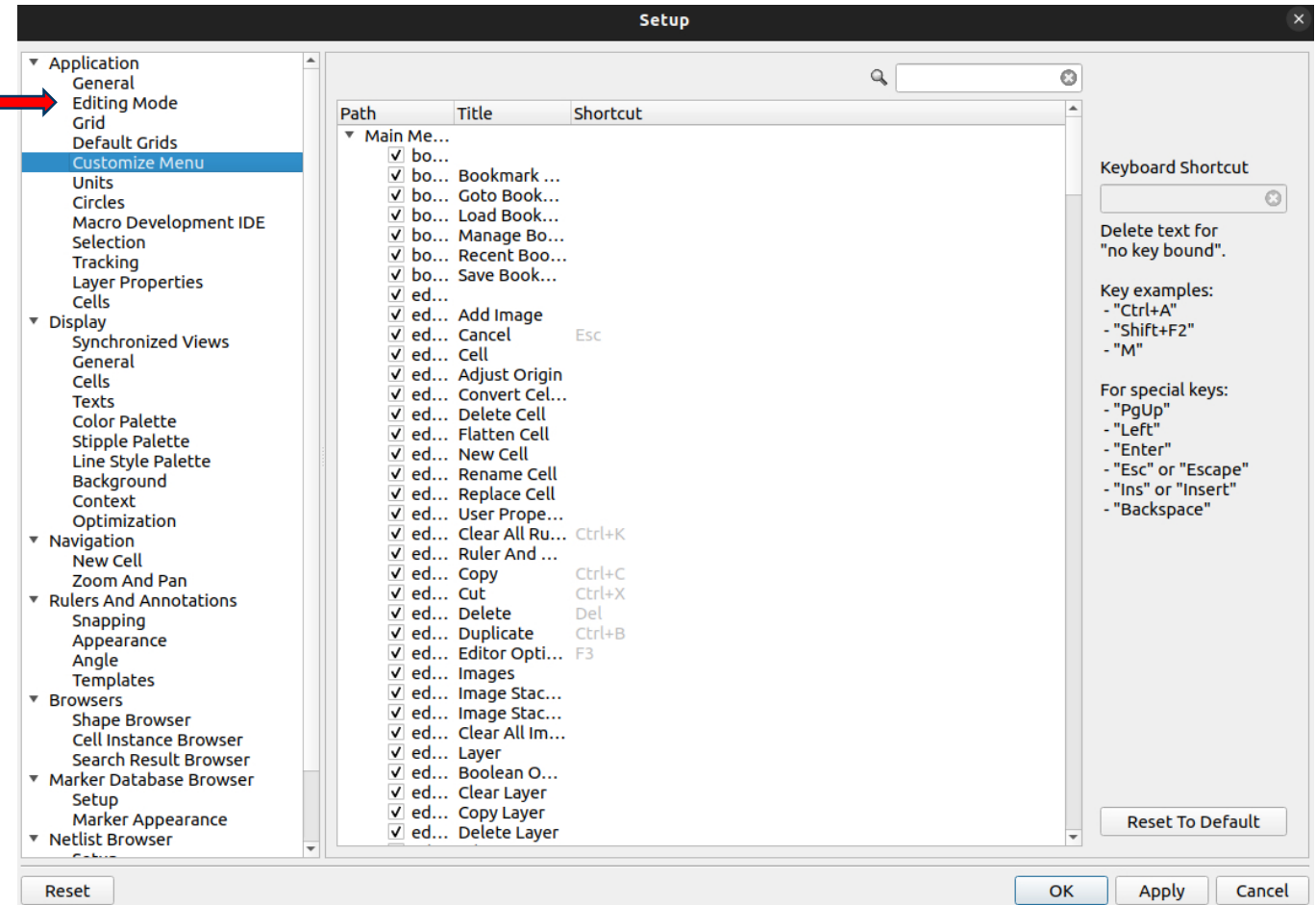
Drag and Drop!

Want to set your own shortcuts?



Possible to set edit mode as default

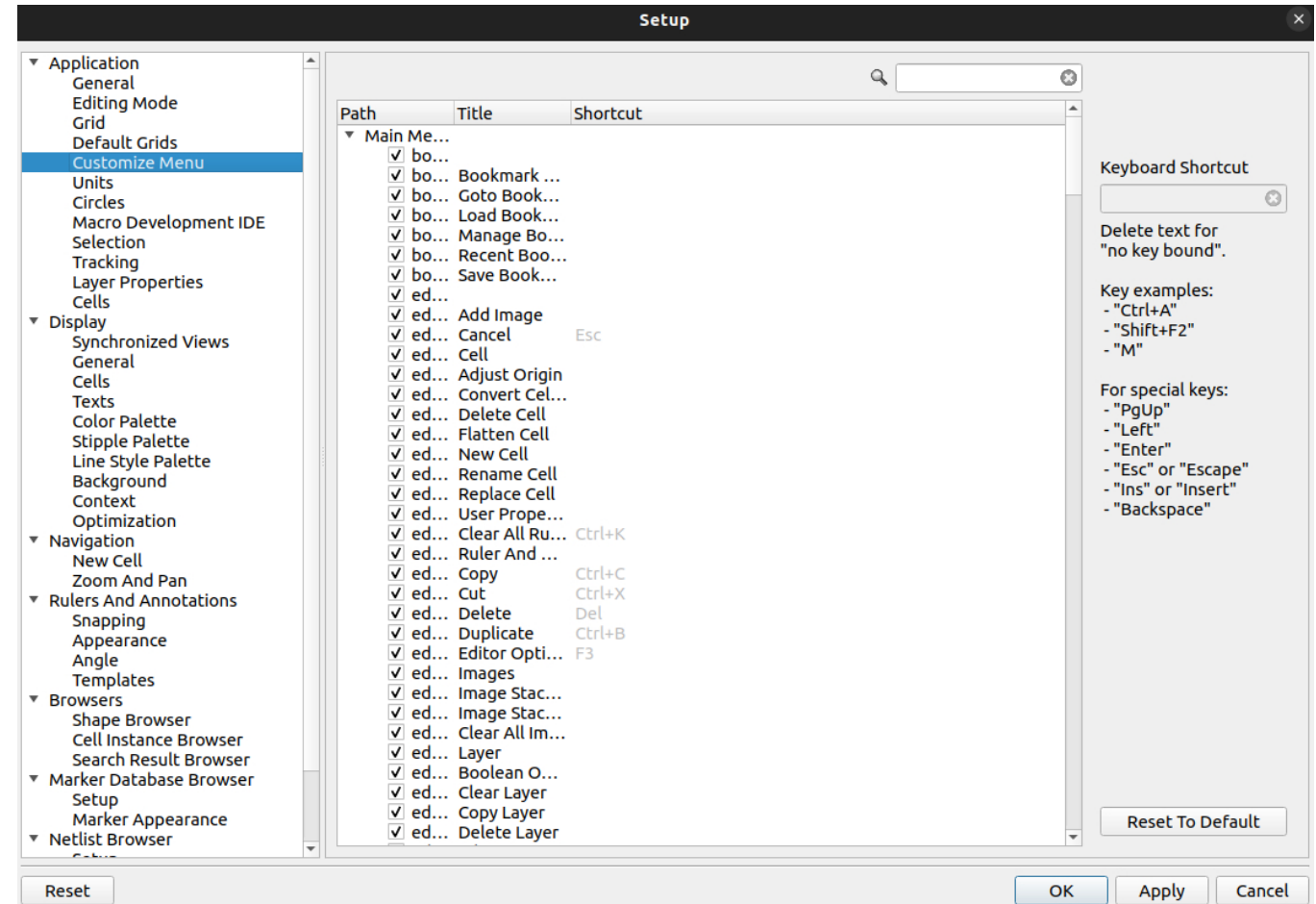
File -> Setup -> Customize Menu



Want to set your own shortcuts?



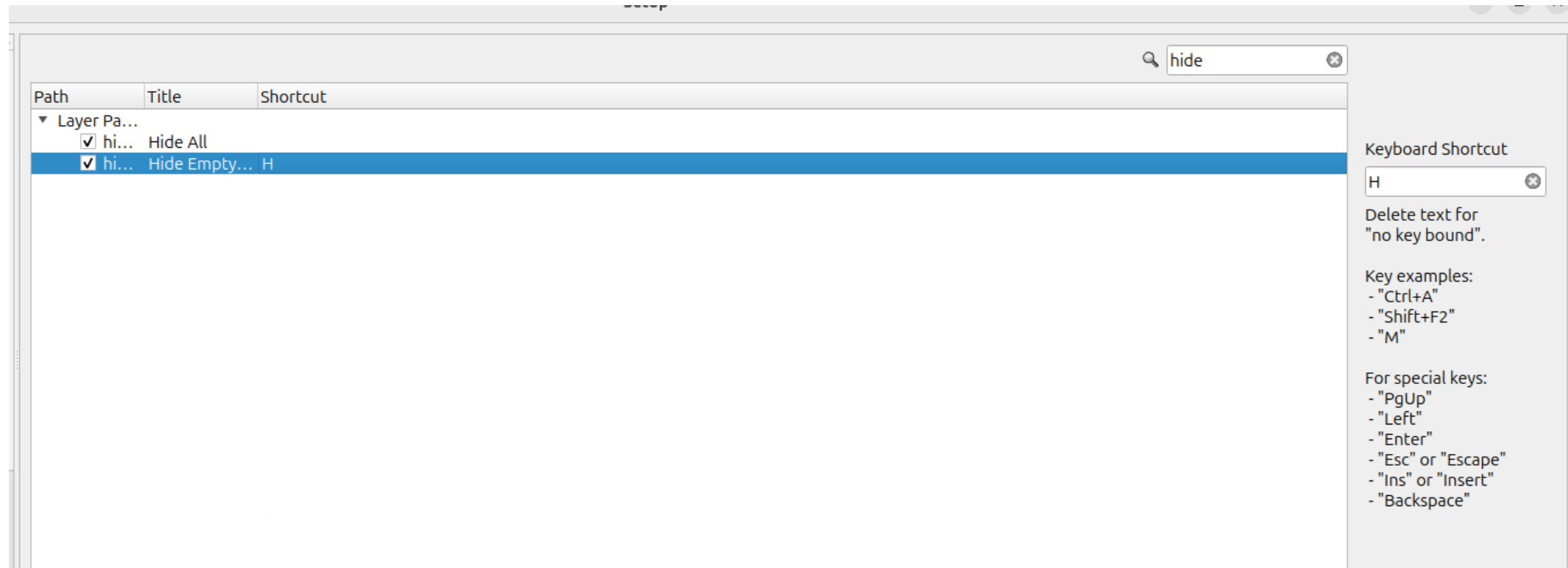
File -> Setup -> Customize Menu



Want to set your own shortcuts?



If shortcuts shown on next slide is not available from default you can set them yourself using the following example below

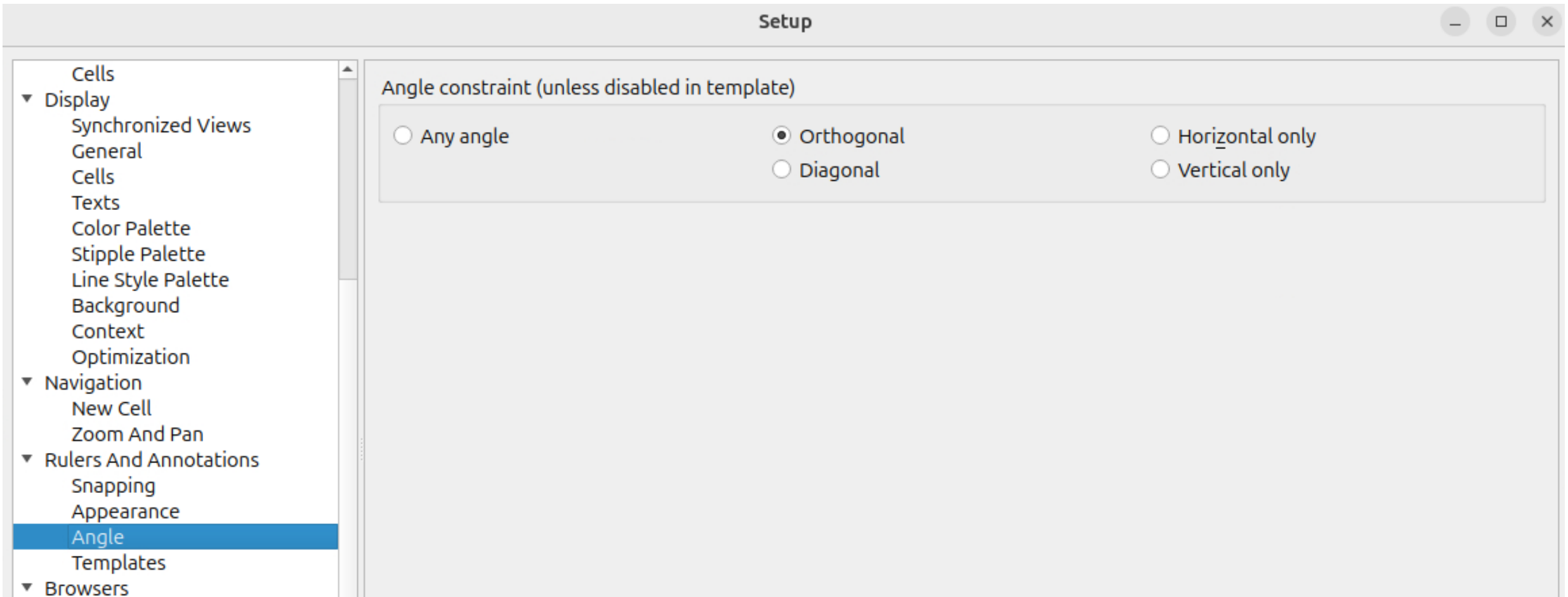


Simple maneuvering

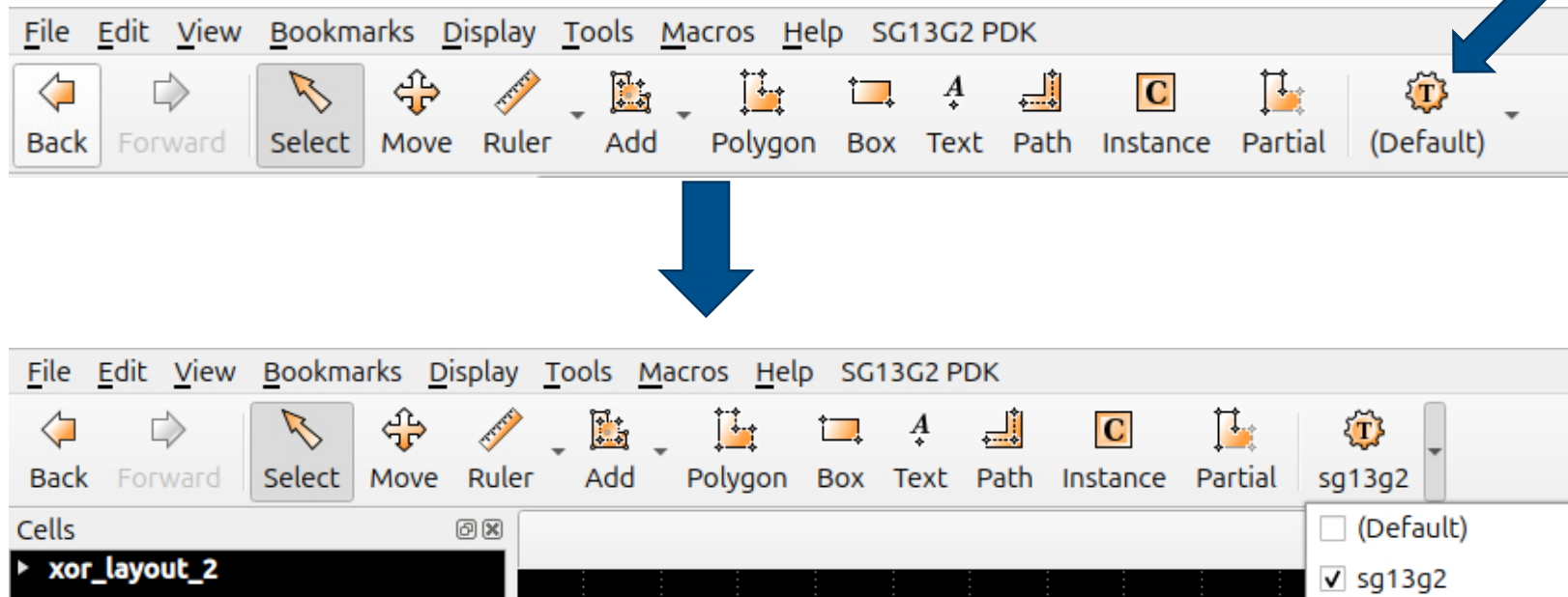


- 0 **Launching Klayout:** klayout -e (edit mode)
- 0 **Zoom Fit (*Main Menu*):** F
- 0 **Move (*Main Menu*):** M
- 0 **Show Only Selected (*layer panel*):** Shift + Tab
- 0 **Show all used layers (*layer panel*):** Tab
- 0 **Hide Empty Layers (*layer panel*):** H
- 0 **Box (*Main Menu*):** W (click desired layer first)
- 0 **Partial (*Main Menu*):** P
- 0 **Ruler (*Main Menu*):** K
- 0 **Clear All Rulers And Annotations (*Main Menu*):** Ctrl + k

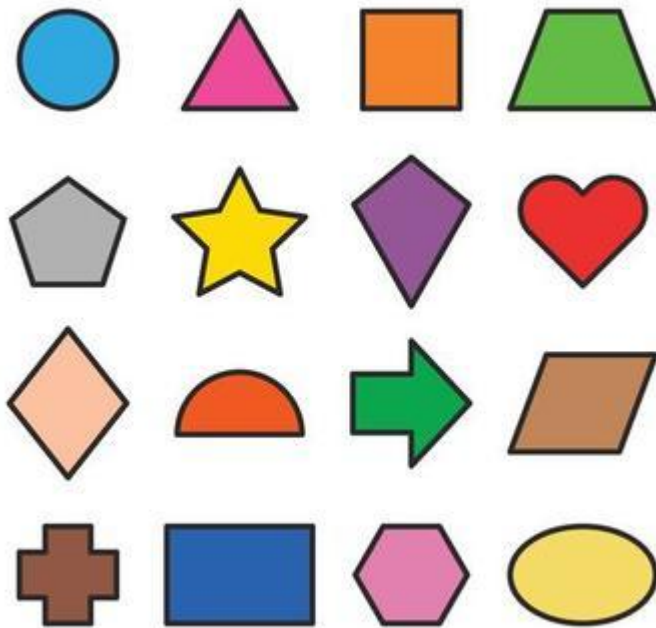
Simple maneuvering



Simple maneuvering



15-20 min playtime: Draw Cool Shapes/View Examples



IHP-AnalogAcademy/utils

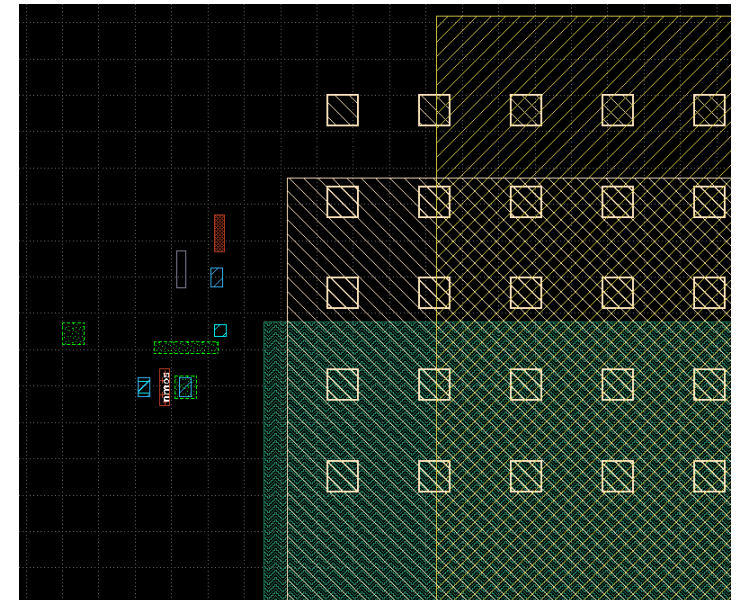
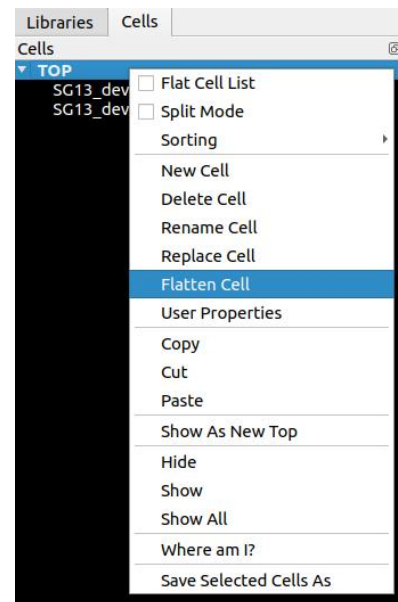
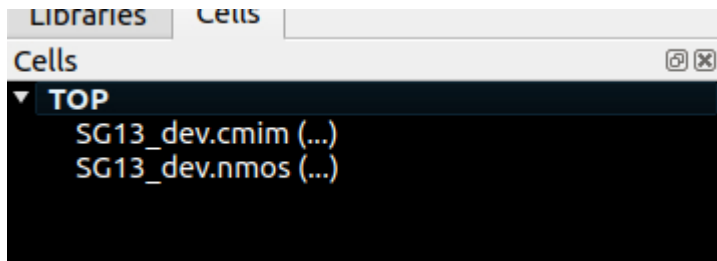
Example from command line: `Layout -e chip_r.gds`

Alternatively: File -> Open -> chip_r.gds

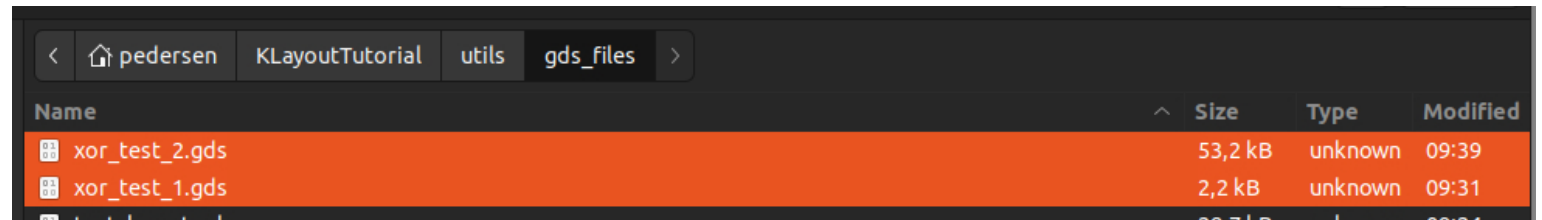
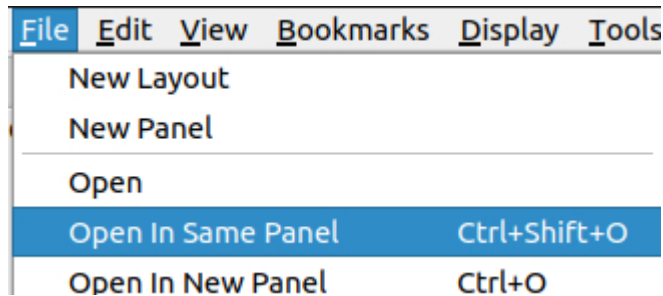
Flattening Your Design



- 0 Combines all instances of sub-cells (child cells) into a single, flat structure, removing the hierarchical design. Why would we need this?
- 0 Prepares the design for export in formats that don't support this specific hierarchy, ensuring compatibility.



- 0 **XOR Function in KLayout:** Performs a bitwise exclusive OR operation between two shapes, returning the area where the shapes differ.
- 0 **Use Cases:** Highlights differences between two layouts or masks. Helps identify discrepancies between design and manufacturing layers.



- 0 In this example its two almost identical designs!

Tools -> XOR Tool

XOR Tool

Input

Layout A: xor_test_1.gds[1], Cell 'layout_1' ▾

Layout B: xor_test_2.gds, Cell 'layout_2' ▾

Use for input: All Layers (different layouts) ▾

From region: All ▾

Options

Compare modes: ☒ A XOR B (differences)
☐ A NOT B (In A but not in B)
☐ B NOT A (in B but not in A)
☐ Summarize missing layers (when output is marker DB)

Tolerances: Tolerances (t1,t2,...) in micron

Hierarchical: ☐ consider hierarchy (experimental)

Tiling: Tile size in micron ☐ Heal result shapes

Threads: 1 Used for tiles and layers

Output

Send output to: Marker database ▾ with offset

Marker Database Browser

Database: XOR (Comparison of 'xor_test_1.gds[1], Cell layout_1' vs. 'xor_test_2.gds, Cell layout_2')

... on layout: xor_test_1.gds[1]

Directory

Cell / Category	Count (Not Visited) - Waived
By Cell	1
By Category	1
XOR	1
All	1

Markers

F	I	W	Marker
%			XOR: 1/0 [layout_1]

Info 1 / 1

1/0 [layout_1]
Results for layer 1/0
polygon: (-1.035,3.895;-1.035,6.58;1.445,6.58;1.445,3.895)

Try with two identical cells to see 0 errors 😊

Running LVS



-0 Open the schematic in the reference (lvs_tester/schematic)

-0 Navigate to the Simulation tab and under LVS select the following ➡

- ✓ LVS netlist + Top level is a .subckt
- ✓ Upper case .SUBCKT and .ENDS
- Top level is a .subckt
- Set 'lvs_ignore' variable
- Use 'spiceprefix' attribute

-0 Then press *Set top level netlist name* and name it with lvs_tester.cdl

-0 Now press netlist. This creates the cdl file and places it in the simulation folder next to the schematic

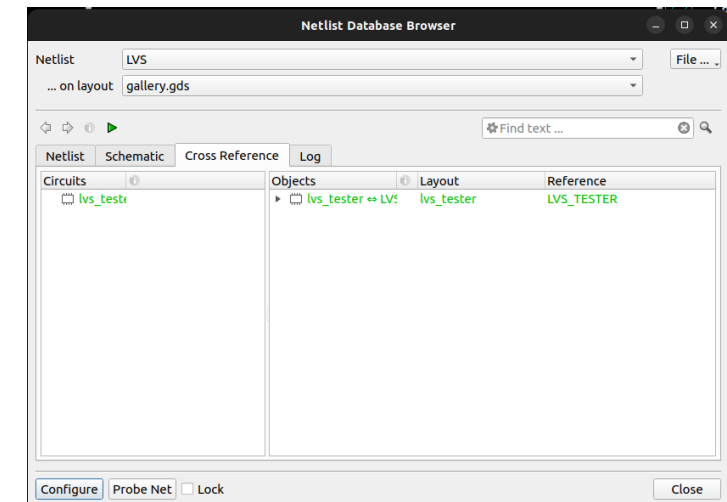
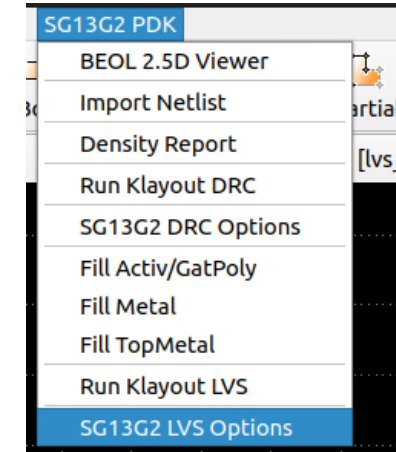
```
.SUBCKT lvs_tester G D gnd
*.PININFO G:B D:B gnd:B
M3 D G gnd sub sg13_lv_nmos w=1.0u l=0.45u ng=1 m=1
R1 gnd sub ptap1 A=6.084e-13 P=3.12e-06
.ENDS
```

Reference: module_0/lvs_tester (provides LVS clean example)

Running LVS



- Now launch Klayout and navigate to SG13G2 PDK -> LVS options
- Browse Netlist path and point to the .cdl file (make sure top cell name is the same as in netlist)
- Now you run the LVS!



Running Parasitic Extraction



- 0 The Parasitic Extraction we will use is called Klayout-pex and is a wrapper around the Magic PEX engine.
- 0 To initialize the environment we will use write in terminal **enter_venv**
- 0 Now try to run **kpex --help**
- 0 **Example of real inputs** ➡
 - **--pdk ihp_sg13g2** — choose technology kit
 - **--gds design.gds** — layout input file
 - **--schematic design.spice** — schematic netlist for LVS
 - **--cell top** — top cell name in GDS
 - **--magic** — use Magic extractor
 - **--magic_mode RC** — extract R+C (or R / C only)
 - **--mode RC** — extraction type for 2.5D engine
 - **--out_dir output/** — results folder
 - **Test** — run on any GDS to verify setup works

Running Parasitic Extraction



- 0 First navigate to modules/module_0_foundations/PEX_Demo/pex
- 0 Open the Kpex.sh shell script

```
#!/bin/bash
set -e

# Hardcoded paths (except PDK stuff)

LAYOUT_PATH="../../layout/inverter.gds"
SCHEMATIC="../../simulations/inverter.spice"

PDK_NAME="ihp_sg13g2"
MAGICRC="$PDK_ROOT/$PDK/libs.tech/magic/ihp-sg13g2.magicrc" # The magicrc file for your PDK, used during extraction

# Run parasitic extraction with kpex
kpex \
  --pdk "$PDK_NAME" \
  --magic \
  --schematic "$SCHEMATIC" \
  --gds "$LAYOUT_PATH" \
  --magicrc "$MAGICRC" \
  --magic_mode CC \
  --magic_cthresh 0.02 \
  --magic_rthresh 50 \
  --magic_short resistor \
  --magic_merge conservative \
  --out dir ./pex_output
```

Running Parasitic Extraction



- 0 Now examine the output of the parasitic extraction found at **pex_output**
- 0 Most importantly *pex_output/inverter__inverter/magic_RCinverter.pex.spice*

* NGSPICE file created from inverter.ext - technology: ihp-sg13g2

```
.subckt inverter Vout Vin Gnd Vdd
X0 Vout Vin Vdd Vdd sg13_lv_pmos ad=0.68p pd=4.68u as=0.68p ps=4.68u w=2u l=0.45u
X1 Vout Vin Gnd Gnd sg13_lv_nmos ad=0.34p pd=2.68u as=0.34p ps=2.68u w=1u l=0.45u
C0 Vout Vdd 0.13155f
C1 Vout Vin 0.10077f
C2 Vin Vdd 0.14482f
C3 Vout Gnd 0.39245f
C4 Vin Gnd 0.64666f
C5 Vdd Gnd 0.15308f
.ends
```

Using The Parasitic Netlist in Simulation



- 0 In order to test the parasitic extracted device we must insert the parasitic netlist
- 0 Navigate to *modules/module_0_foundations/PEX_Demo* and open *inverter_test.sch*
- 0 *Here you will find a schematic with a symbol referencing to no schematic!*
- 0 Double click the symbol and write the following inside: ➡
name=x1
schematic=inverter
spice_sym_def=""
- 0 And follow this by the parasitic netlist, and then run the sim

```
name=x1
schematic=inverter
spice_sym_def=""
* NGSPICE file created from inverter.ext - technology: ihp-sg13g2

.subckt inverter Vout Vin Gnd Vdd
X0 Vout Vin Vdd Vdd sg13_lv_pmos ad=0.68p pd=4.68u as=0.68p ps=4.68u w=2u l=0.45u
X1 Vout Vin Gnd Gnd sg13_lv_nmos ad=0.34p pd=2.68u as=0.34p ps=2.68u w=1u l=0.45u
C0 Vout Vdd 0.13155f
C1 Vout Vin 0.10077f
C2 Vin Vdd 0.14482f
C3 Vout Gnd 0.39245f
C4 Vin Gnd 0.64666f
C5 Vdd Gnd 0.15308f
.ends
"
```

Using The Parasitic Netlist in Simulation

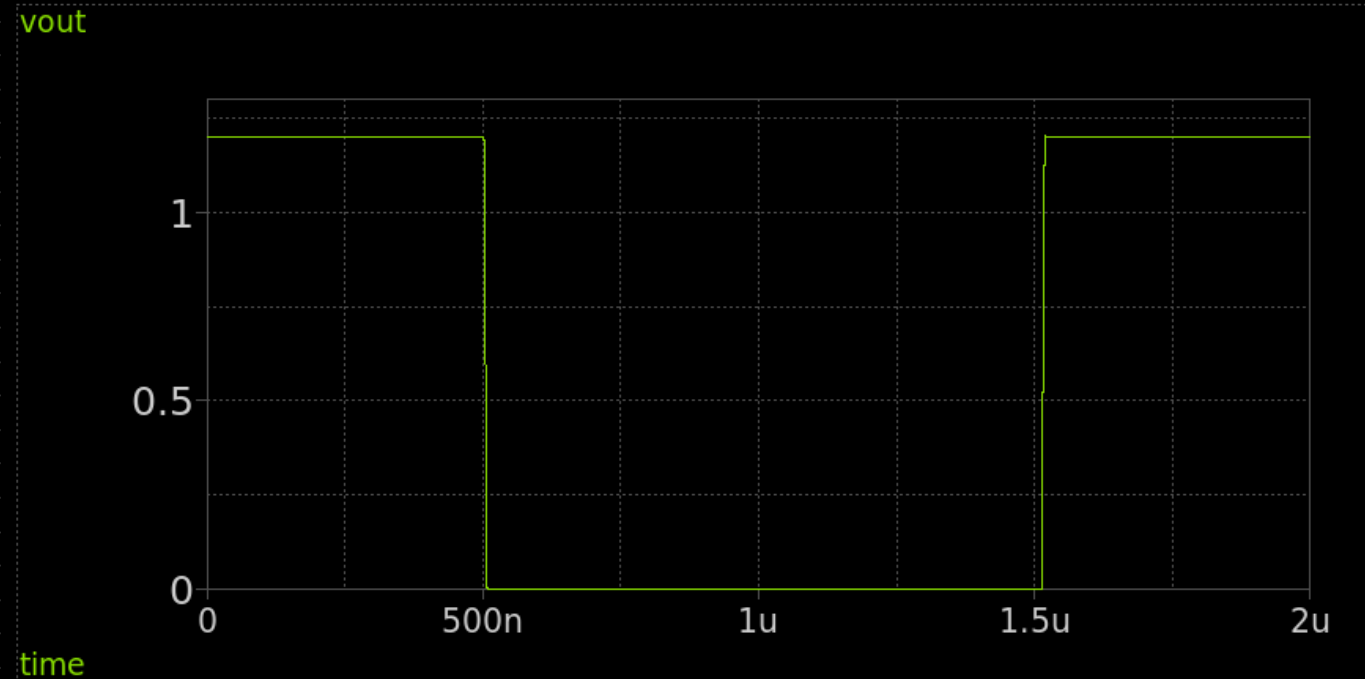
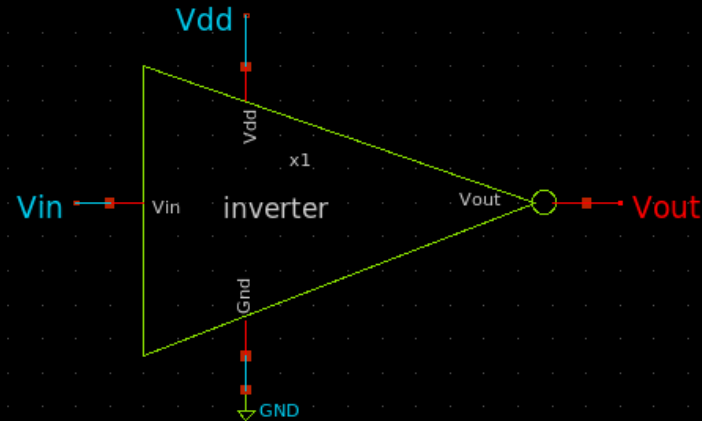


NGSPICE

```
.control  
save all  
tran 50n 2u  
write test_inverter.raw  
.endc
```

MODEL

```
.lib cornerMOSlv.lib mos_tt  
.lib cornerRES.lib res_tpy
```



➡ load waves



What Will We Do Tomorrow?



- 0 **OTA Design & Analysis:** Perform DC and AC analysis to evaluate key design metrics.
- 0 **Bandgap Voltage Reference:** Conduct DC, transient, and mismatch analysis using Monte Carlo sampling.
- 0 **From Design to Tape-Out:** Explore the final layout and discuss the steps required for tape-out in the open-source domain.
- 0 **Layout :** Exploring layout of the OTA

Hands On Session!!



- 0 **Complete the Schematic (Inverter):** Finalize the inverter schematic and explore Xschem simulations. For reference, consult the Ngspice documentation: ngspice.sourceforge.io/docs/ngspice-manual.pdf
- 0 **From Simulation to Layout:** Transition from simulation to creating a basic layout of the inverter. (Assistance will be provided during the session). Physical verification is not a must at this stage
- 0 **Prepare for Tomorrow's Session:** Optionally, begin working on tasks for tomorrow's session! Navigate to:

IHP-AnalogAcademy/modules/module_1_bandgap_reference/part_1_OTA/design_of_ota.md